Lecture 5

GPUs

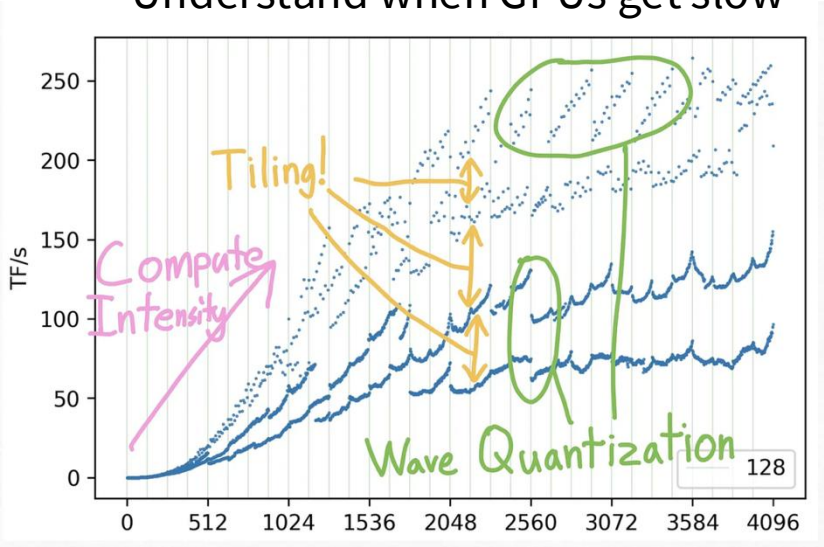
CS336

Tatsu H

Outline and goals

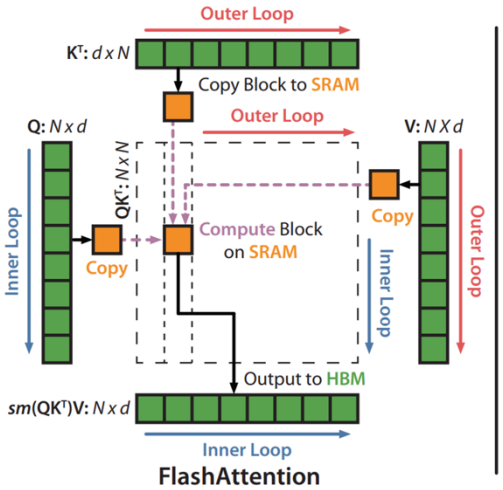
- Make CUDA and GPUs less magic

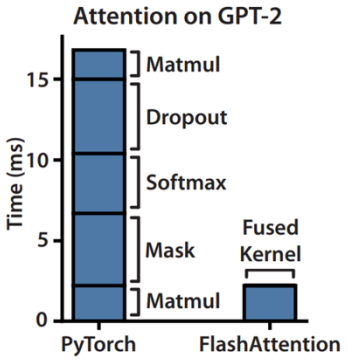
Understand when GPUs get slow



https://www.thonking.ai/p/what-shapes-do-matrix-multiplications

Understand how to make fast algorithms

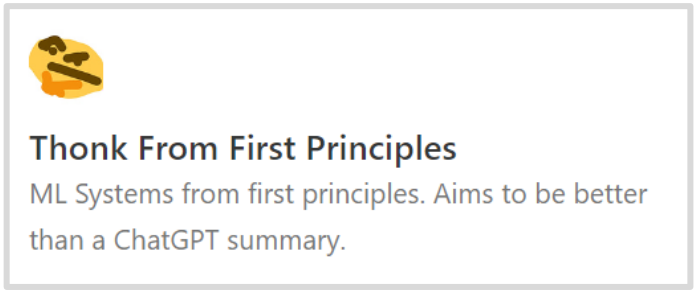


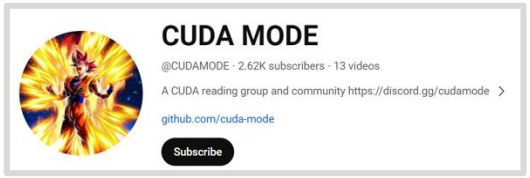


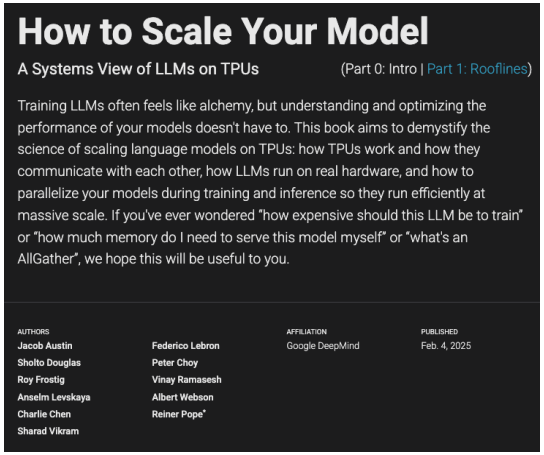
Dao et al, Flash Attention

Before we start..

Substantial credit goes to a few sources that I'd like to highlight..







Horace He's blog

CUDA Mode group

TPU (and now GPU) book (!!)

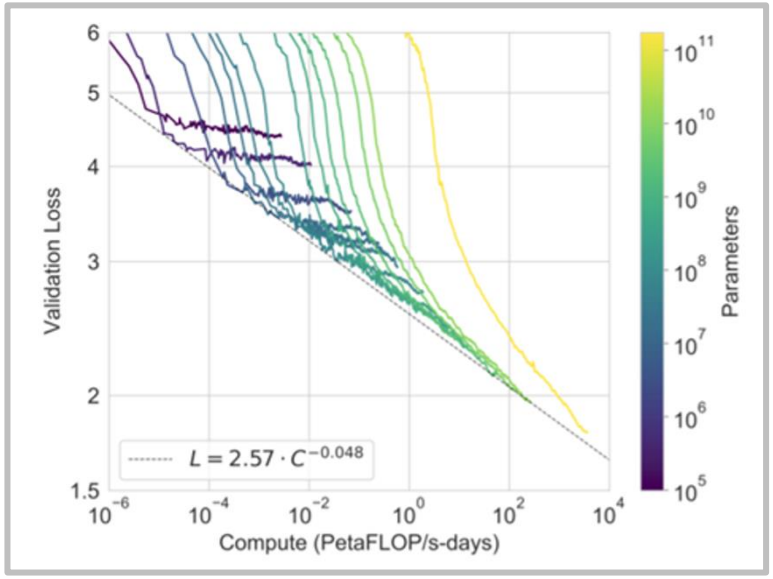
And other sources including https://nichijou.co/, https://jonathan-hui.medium.com/

Organization today:

- **Part 1**: GPUs in depth – how they work and important parts
- **Part 2**: Understanding GPU performance
- **Part 3**: Putting it together – unpacking FlashAttention

Setting the stage: compute leads to predictable perf

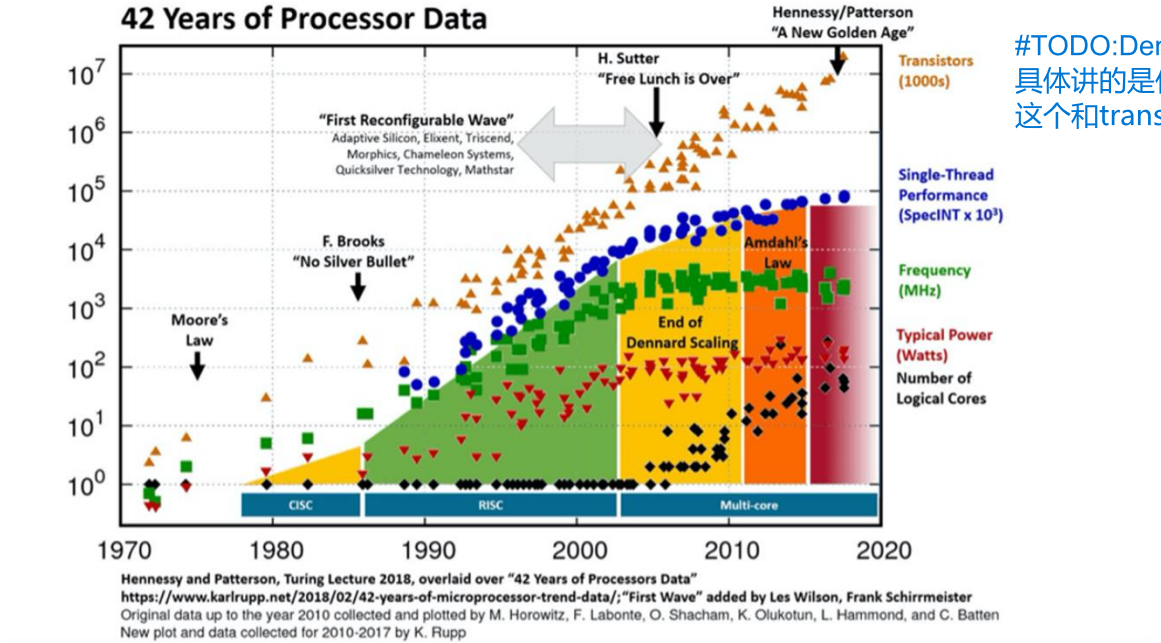
Often times, compute leads to predictable performance gains for language models



Kaplan et al, Neural Scaling Laws

Faster hardware, better utilization, improved parallelization alone can drive progress (for now..)

How do we get compute scaling? Early on – Dennard scaing

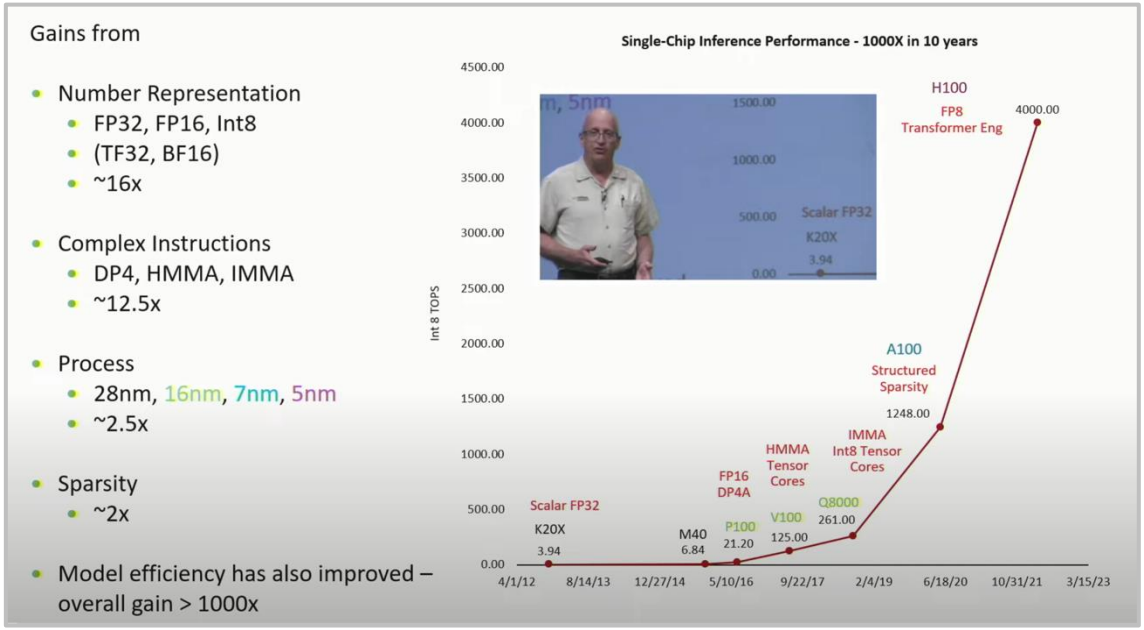


#TODO:Dennard scaling
具体讲的是什么？clock speed是什么？
这个和transistors的数量、大小有关系吗？

But the traditional form of scaling (*Dennard scaling*) from 1980-2000s has tapped out.

.. How do we feed LLMs' insatiable appetite for compute?

Parallel scaling continues



flops/s发展得很快

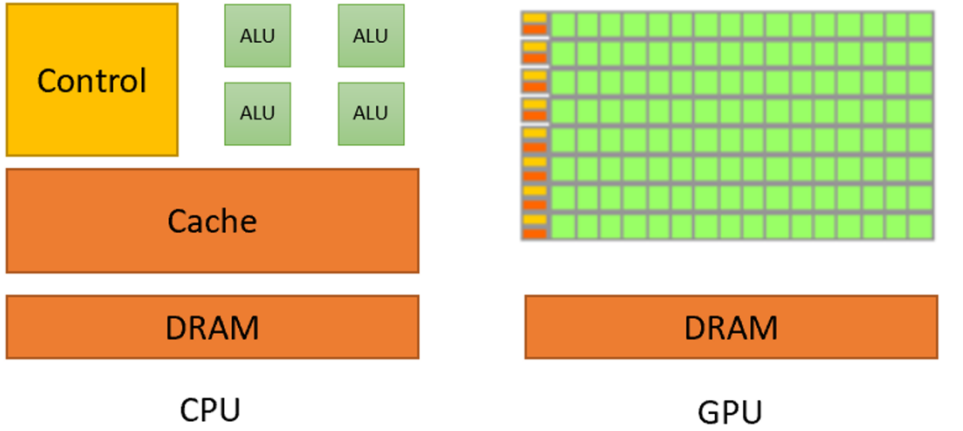
Bill dally, HotChips keynote

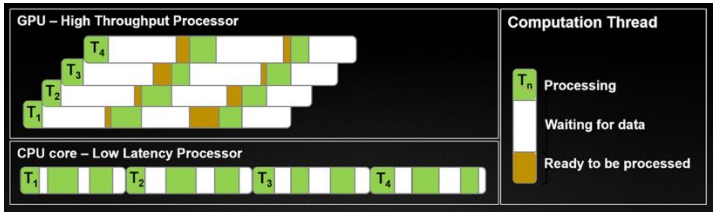
Parallel scaling with GPUs has scaled > 1000x in 10 years.

There is no LLM scaling without GPU scaling

How is a GPU different from a CPU?

CPUs optimize for a few, fast threads while GPUs optimize for many many threads





Many tiny compute units (ALUs).
Much less support for branching (control, cache)

#TODO cpu is designed for quick serial execution，如何理解？和gpu的设计不同点？cpu有着complext control flows，又如何理解？control units和ALU分别是啥？

CPUs optimize for latency (each thread finishes quickly)
GPUs optimize for throughput (total processed data)

#TODO 这里说的CPU和GPU两者optimize for的不同东西，具体是什么含义？throughput是啥？

https://developer.nvidia.com/blog/cuda-refresher-reviewing-the-origins-of-gpu-computing/

Anatomy of a GPU (execution units)



SM

这里可以结合https://zhuanlan.zhihu.com/p/194345394141423626 0
这个文章理解



GA100 Full GPU with 128 SMs

#TODO shared memory和L1缓存的作用区别？ shared memory能被programming?

Each SM further contains many **SPs (streaming processor)** that can execute ‘threads’ in parallel

一个SM内可以并行多个线程

GPUs have *many* **SM (streaming multiprocessors)** that independently execute ‘blocks’ (jobs).

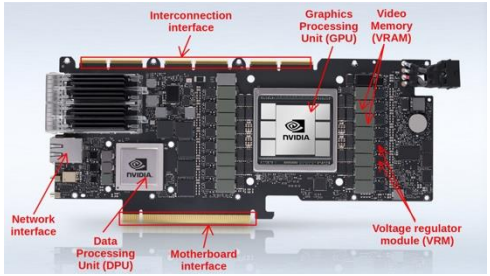
Anatomy of a GPU (memory)

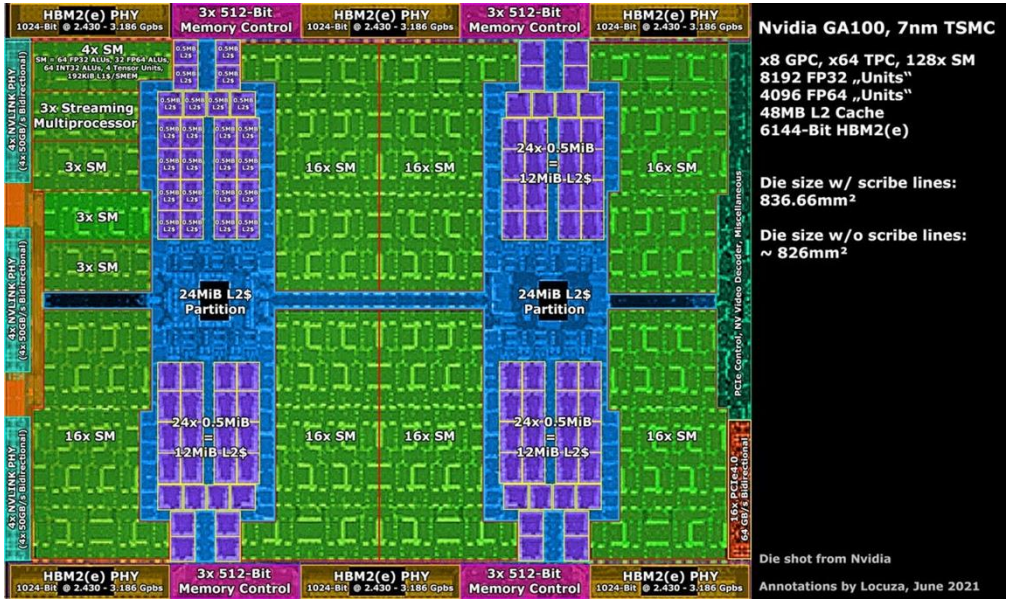
global memry指的就是HBM，也就是常说的显存

The closer the memory to the SM, the faster it is – L1 and shared memory is *inside* the SM. L2 cache is on die, and global memory are the memory chips next to the GPU

TABLE IV
THE MEMORY ACCESSES LATENCIES

Memory type	*CPI (cycles)*
Global memory	290
L2 cache	200
L1 cache	33
Shared Memory (ld/st)	(23/19)





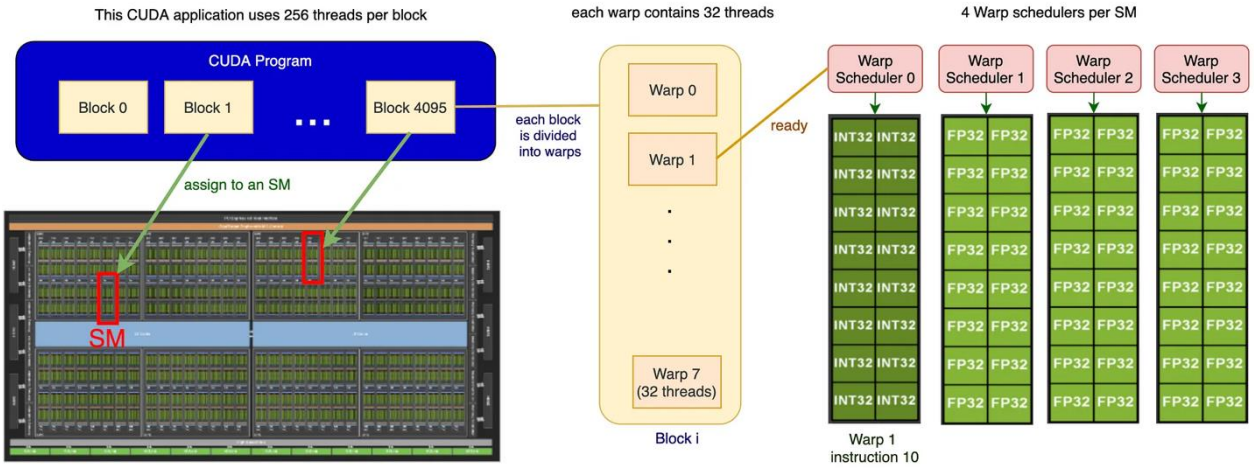
L2在SM外部，L1在SM内部

#TODO SRAM和DRAM全称是什么?

SRAM (shared/cache memory) is much more expensive (100x) but ~ 8x faster than DRAM (Global memory)

Execution model of a GPU

#TODO：shared memory和L1 cache的区别是啥?
为什么说we don't get control of L1 cache?



这里指的是一个warp里的所有threads都执行相同指令，而不是一个block里的

There are 3 important players in the execution model

Threads: Threads 'do the work' in parallel – all threads execute the same instructions but with different inputs (SIMT).

threads并行地执行指令，每个block都有着自己的shared memory。

Blocks: Blocks are groups of threads. Each block runs on a SM w/ its own shared memory.

一个block可以拆分为多个warp

Warp: Threads always execute in a 'warp' of 32 consecutively numbered threads each.

Warp是32 个线程的集合，是 GPU 的最小调度粒度。

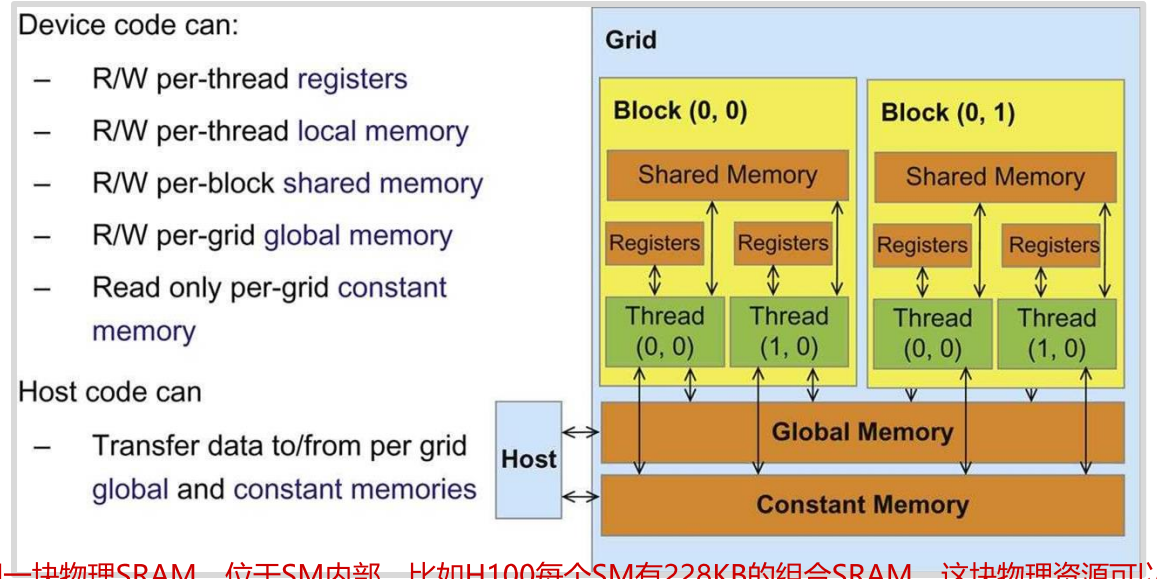
shared memory和L1 cache都位于每个SM内部（每个SM一套）

Memory model of a GPU

registers：寄存器。寄存器是线程私有的存储，访问速度最快，延迟极低，但容量有限，每个线程只能使用少量寄存器

寄存器存的都是最底层的，比如memory address，标识了你需要的array的start和end

一旦需要访问global memory，
那么速度就很慢了



每个线程都有自己私有的寄存器。更准确地说，是每个线程获得一组寄存器的分配，而不是物理上独占的。

每个block都有各自的shared memory，但同样是物理资源层面共享、逻辑上独立分配的关系。

这里的Global Memory指的就是显存，就是DRAM。

共享内存和L1缓存共用同一块物理SRAM，位于SM内部。比如H100每个SM有228KB的组合SRAM，这块物理资源可以在共享内存和L1缓存之间灵活划分比例。每个SM内部都有一块shared memory。图中的这两块Shared Memory是从逻辑意义上看的。物理意义上来看是只有一块。
每个Thread Block在SM上启动时，会从这块SRAM里分配出一部分作为自己的共享内存。关键点：

Each thread can access its own register, and shared memory within the block.

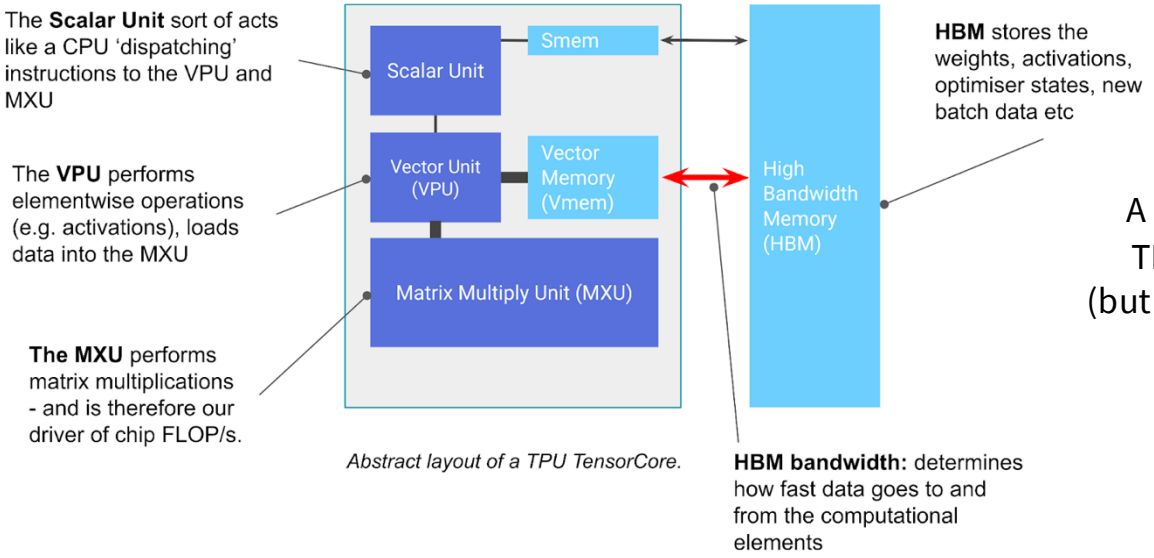
Information that goes across blocks need to be read/written to global memory (slow)

Block间隔离：不同Block的共享内存是逻辑隔离的，Block A看不到Block B的共享内存内容
Block内共享：同一个Block内的所有线程可以读写同一块共享内存，这就是"shared"的含义
生命周期：Block执行完毕后，它占用的共享内存自动释放，还给SM分配给下一个Block

Side thread – What about TPUs?

#TODO：TPU相比GPU的主要区别?

GPUs, TPUs, and many other accelerators are at a high level, similar



Abstract layout of a TPU TensorCore.

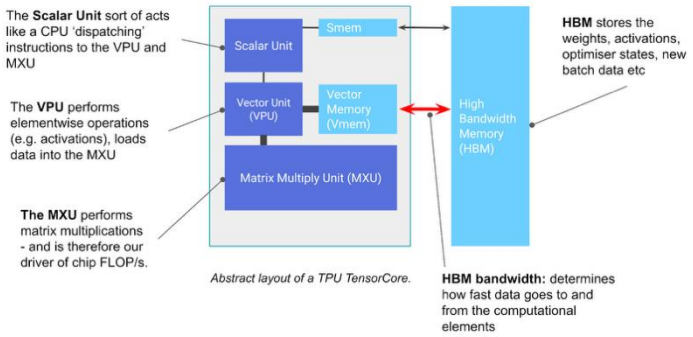
A GPU has *more SMs*
TPUs has fewer TCs
(but similar matmul perf)

Core structure – lightweight control, fast (big) matmul unit, fast memory.

Differences - how the accelerators are networked (in the parallelism lecture)
- no warps (just blocks – tradeoffs in matmul vs non-matmul)

TPU和GPU底层的核心想法是一致的，知识也是可以迁移的

Side thread – What about TPUs?



Abstract layout of a TPU TensorCore.

GPU	TPU	What is it?
Streaming Multiprocessor (SM)	Tensor Core	Core "cell" that contains other units
Warp Scheduler	VPU	SIMD vector arithmetic unit
CUDA Core	VPU ALU	SIMD ALU
SMEM (L1 Cache)	VMEM	Fast on-chip cache memory
Tensor Core	MXU	Matrix multiplication unit
HBM (aka GMEM)	HBM	High bandwidth high capacity memory

TPU：MXU更大，但是数量更少

GPU	TPU	H100 #	TPU v5p #
SM (streaming multiprocessor)	Tensor Core	132	2
Warp Scheduler	VPU slots	528	8
SMEM (L1 cache)	VMEM	32MB	128MB
Registers	Vector Registers (VRegs)	32MB	256kB
Tensor Core	MXU	528	8

两者的tensor cores定义不同

GPU中，tensor core
指的是matrix multipy units
这是SM内部的矩阵运算单元，
专门执行小矩阵乘法。
半精度下执行16x16x16尺寸的小矩阵乘法。
需要计算时，被Warp调用。

Core structure – lightweight control, fast (big) matmul unit, fast memory.

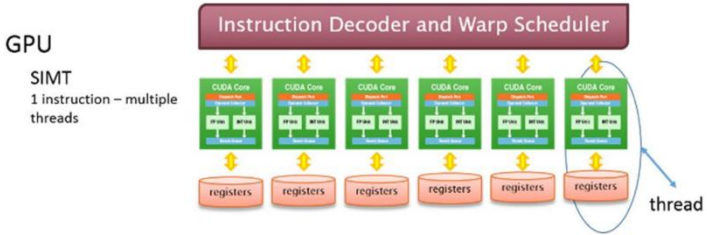
https://jax-ml.github.io/scaling-book/gpus/

Strengths of the GPU model

- Easily scales up hard workloads (by adding more SMs)

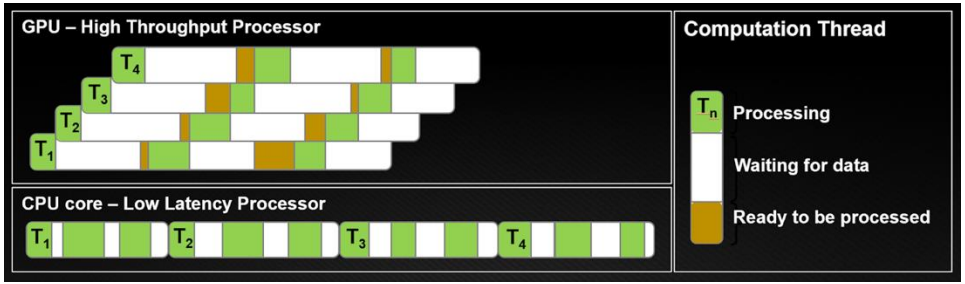
- Easy (?) to program due to the SIMT model

#TODO：这里的SIMT model是啥？为啥让program变得简单了？

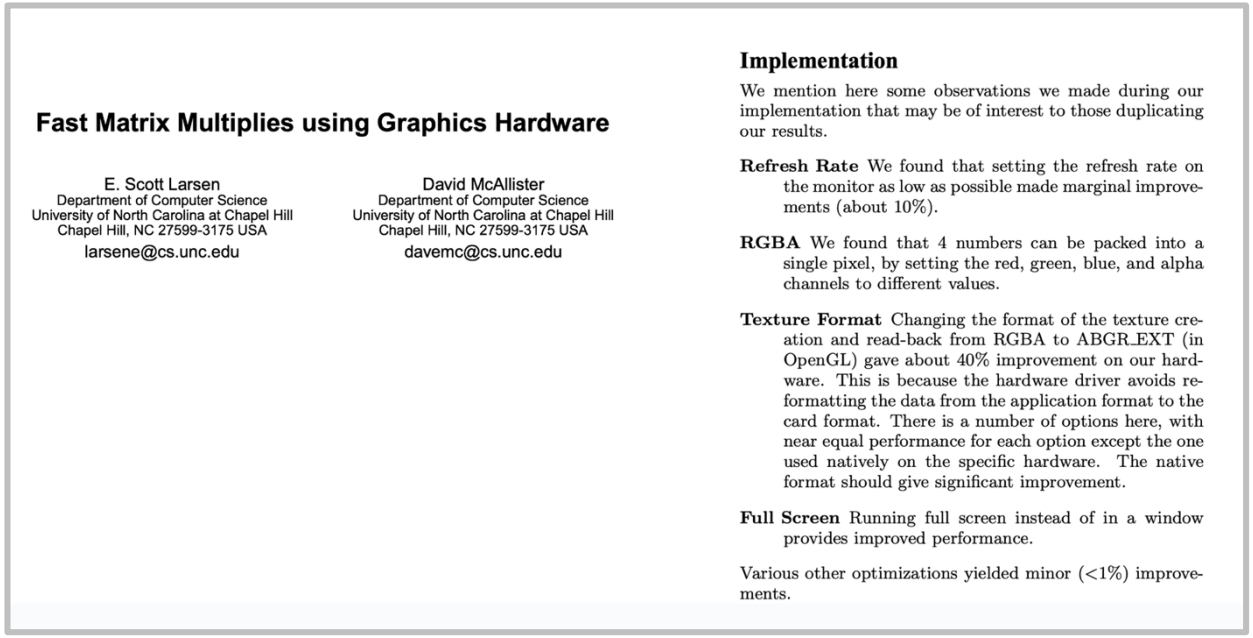


- Threads are ‘lightweight’ and can be stopped and started

这样可以增加throughput（吞吐量）



GPUs as fast matrix multipliers



Fast Matrix Multiplies using Graphics Hardware

E. Scott Larsen
Department of Computer Science
University of North Carolina at Chapel Hill
Chapel Hill, NC 27599-3175 USA
larsene@cs.unc.edu

David McAllister
Department of Computer Science
University of North Carolina at Chapel Hill
Chapel Hill, NC 27599-3175 USA
davemc@cs.unc.edu

Implementation

We mention here some observations we made during our implementation that may be of interest to those duplicating our results.

Refresh Rate We found that setting the refresh rate on the monitor as low as possible made marginal improvements (about 10%).

RGBA We found that 4 numbers can be packed into a single pixel, by setting the red, green, blue, and alpha channels to different values.

Texture Format Changing the format of the texture creation and read-back from RGBA to ABGR_EXT (in OpenGL) gave about 40% improvement on our hardware. This is because the hardware driver avoids reformatting the data from the application format to the card format. There is a number of options here, with near equal performance for each option except the one used natively on the specific hardware. The native format should give significant improvement.

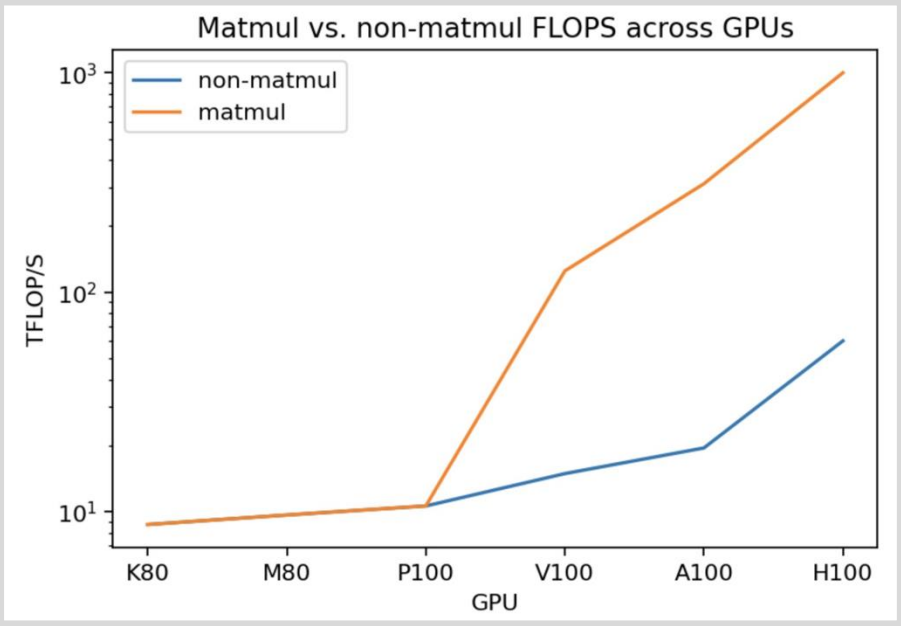
Full Screen Running full screen instead of in a window provides improved performance.

Various other optimizations yielded minor (<1%) improvements.

Early days of NVIDIA GPUs – programmable shaders. Researchers hacked this to do matmuls

最早GPU不是为了矩阵乘法准备的，所以这个work is kind of hacky

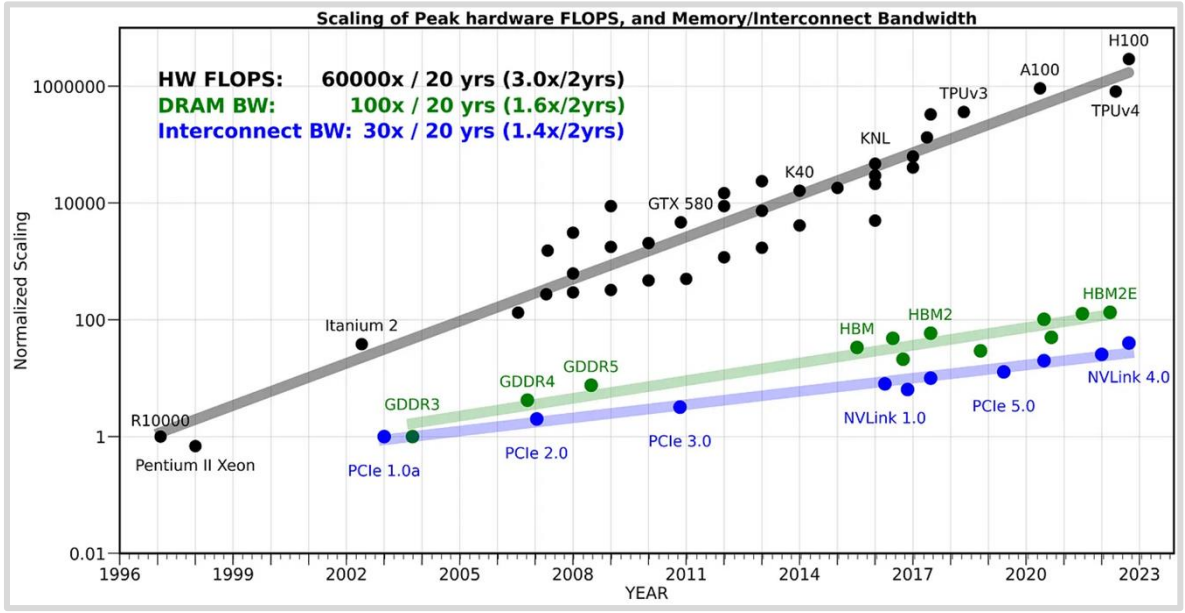
New matmul hardware means matmuls are fast and special



Tensor cores (introduced in V, T series) are specialized matrix multiplication circuits.

Matmuls are >10x faster than other floating point ops!

Compute scaling is faster than memory scaling



绿色：memory bandwidth，即从memory运输数据到算子的速度

紫色：
speed of connecting different GPU devices

https://medium.com/riselab/ai-and-memory-wall-2cb4265cb0b8

这些年发展，机器能执行的flops速度，相比memory带宽的增幅大了很。这也是我们容易memory-bound的原因吧。

FLOPs scale faster than memory – it's hard to keep our compute units fed with data!

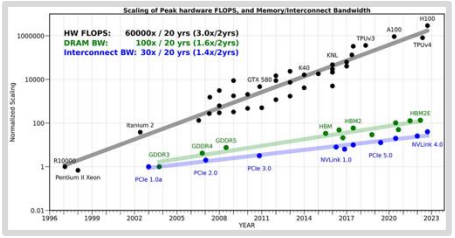
所以我们需要做很多memory层面的优化

Recap: GPUs – what are they and how do they work

- GPUs are massively parallel – same instructions applied across many workers



- Compute (and esp matmuls) have scaled faster than memory



- We have to respect the memory hierarchy to make things go fast.

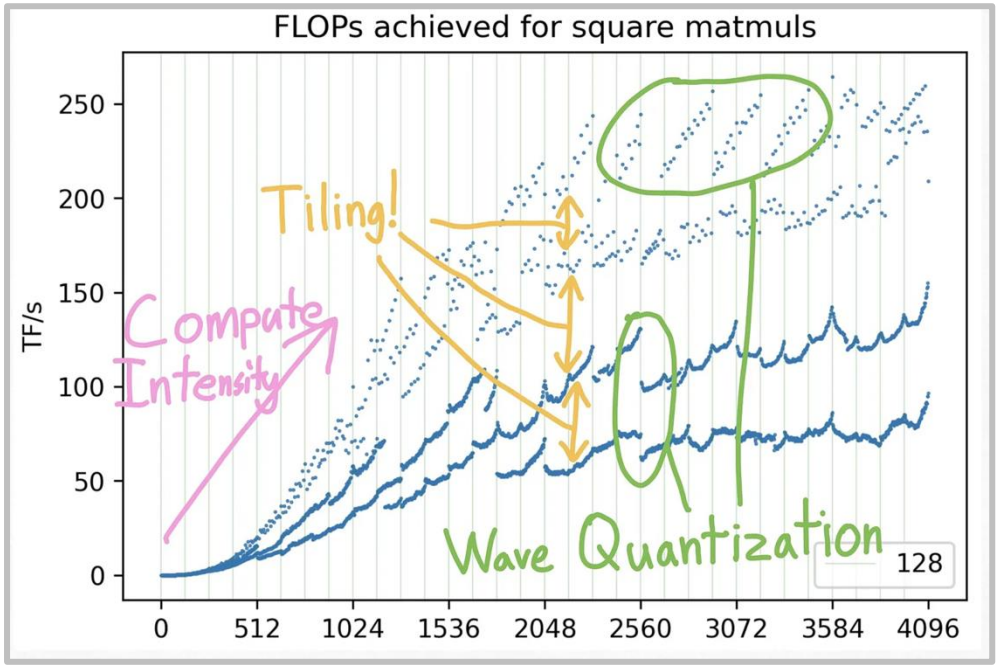
  希望尽可能少地“用到”global memory，
  尽可能让事情在shared memory中完成

TABLE IV
THE MEMORY ACCESSES LATENCIES

Memory type	*CPI (cycles)*
Global memory	290
L2 cache	200
L1 cache	33
Shared Memory (ld/st)	(23/19)

Part 2: Making ML workloads fast on a GPU

纵轴就是throughput，表示每秒钟进行的运算次数

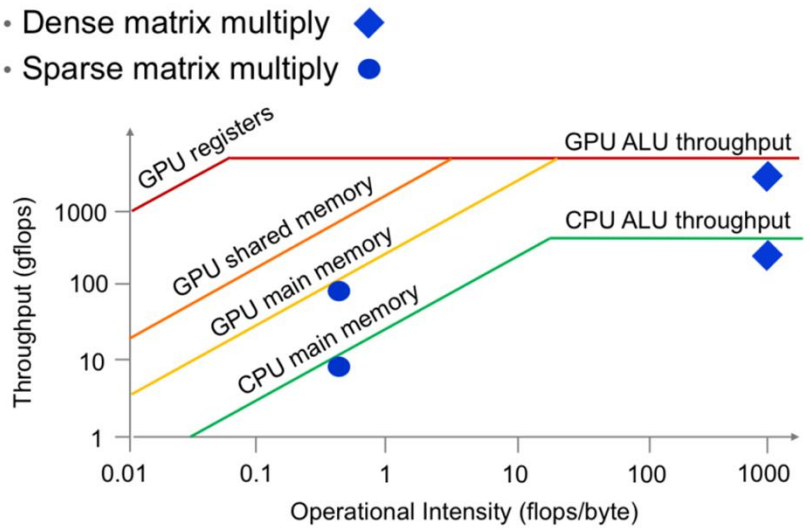


Performance on a GPU can be complex, even for something as simple as a square matmul

What makes ML workloads fast?

#TODO：throughput实际的定义，是不是就是flops/s，也就是MFU计算公式的分子，代表实际每秒的运算量？

The roofline model



对角斜线上：memory-bound
后面的水平线上：compute-bound
（我们的目标）

Key to this section: **how do we avoid being memory bound?**

How do we make GPUs go fast?

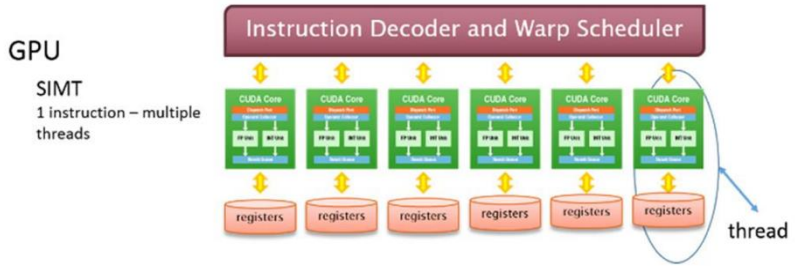
1. **Control divergence (not a memory bottleneck..)**
2. **Low precision computation**

2-6都是想要avoid memory movement amsp

3. **Operator fusion**
4. **Recomputation**
5. **Coalescing memory**
6. **Tiling**

Control divergence (not a memory issue)

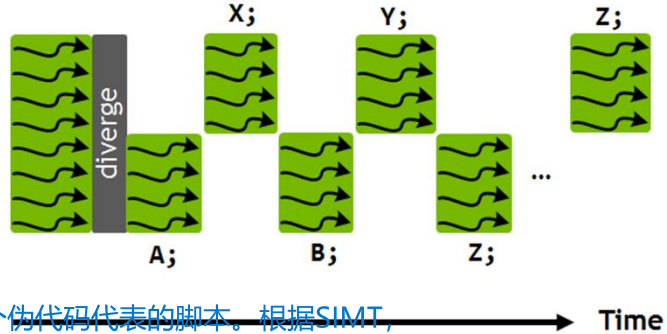
GPUs operate in a SIMT model – every thread in a warp is executing the same instruction



Conditionals are fine, but lead to significant overhead from the execution model

#TODO 详细解释下这个例子
我不理解为啥需要同时
计算AB和XY两个分支，而不是
根据具体代码条件，只需要
计算一个分支即可?

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```

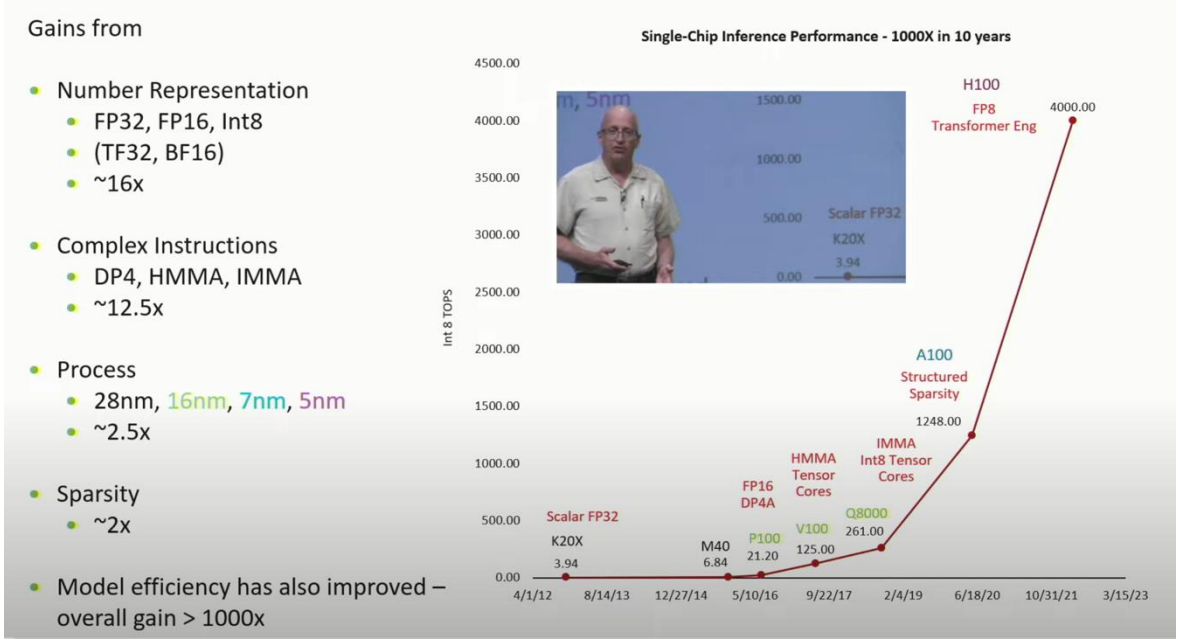


这个图例指的是在
GPU上运行一个
if statement

图例上下各代表一部分threads
则在同一个时间，warp中
的这两部分threads都需要
做一件事情，所以一个
时间会有一半的threads被
mask掉

这个例子可以理解为：让一个warp（包含32个threads），执行这个伪代码代表的脚本。根据SIMT，
同一时刻只能执行同一条指令，而且不同thread的输入不同（这里用threadidx替代），
所以很正常的情况是不同thread走的分支不同，一些走if，一些走else。

Trick 1: Low precision computation



If you have fewer bits, you have fewer bits to move

#TODO：这里要如何理解？low precision能一定程度上缓解memory-bound，所以能够增大推理速度？那么这个不是也是一种取舍吗？相当于放弃了计算的精度？

Low precision improves arithmetic intensity

Example: elementwise ReLU ($x = \max(0, x)$) on a vector of size n**.**

(Float 32 case)

Memory access: 1 read (x), 1 write (if x < 0), float 32 = 4 bytes per op

Operations: 1 comparison op, 1 FLOP.

Intensity: 8 bytes / FLOP

(Float 16)

Memory access: 1 read (x), 1 write (if x < 0), float 16 = 2 bytes per op

Operations: 1 comparison op, 1 FLOP.

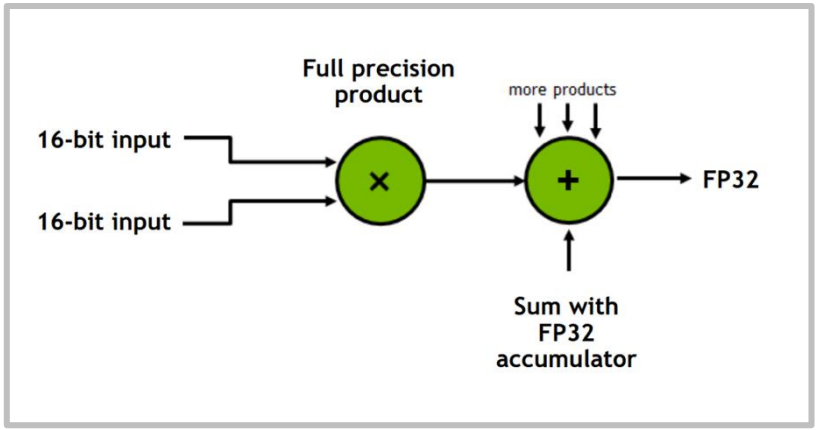
Intensity: 4 bytes / FLOP

Low precision drives faster matrix multiplies

Lots of operations in modern GPUs are accelerated via low / mixed precision operations

#TODO 这个图中的混合精度计算具体为啥要这样做？为什么有时候是16bit，有时候是32bit?

Tensor cores



Operations that can use 16-bit storage (FP16/BF16)

- Matrix multiplications
- Most pointwise operations (e.g. relu, tanh, add, sub, mul)

Operations that need more precision (FP32/FP16)

- Adding small values to large sums can lead to rounding errors
- Reduction operations (e.g. sum, softmax, normalization)

Operations that need more range (FP32/BF16)

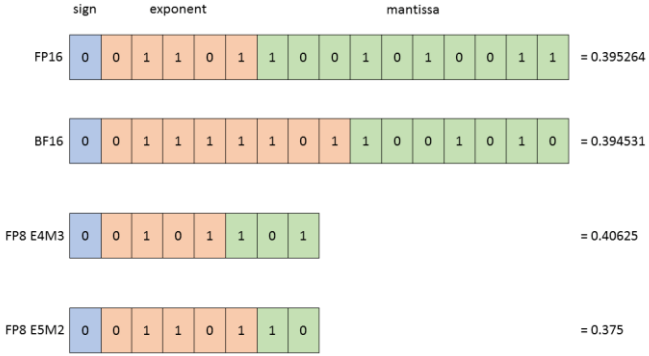
- Pointwise operations where $|f(x)| \gg |x|$ (e.g. exp, log, pow)
- Loss functions

https://nvlabs.github.io/eccv2020-mixed-precision-tutorial/files/dusan_stosic-training-neural-networks-with-tensor-cores.pdf

Frontiers in low precision

#TODO：这里完全没看懂。为啥transpose matrix出现了问题？出现了什么问题？如何解决的？

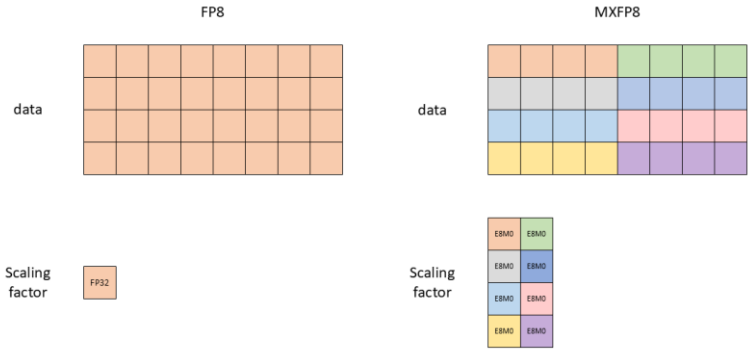
Very low precision (FP8) with different tradeoffs

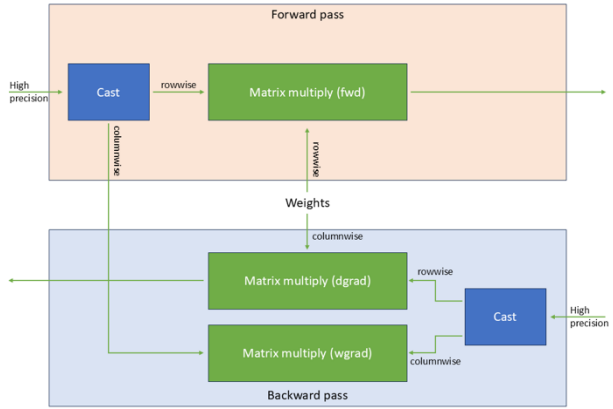


MXFP8 has many *interesting* things about it

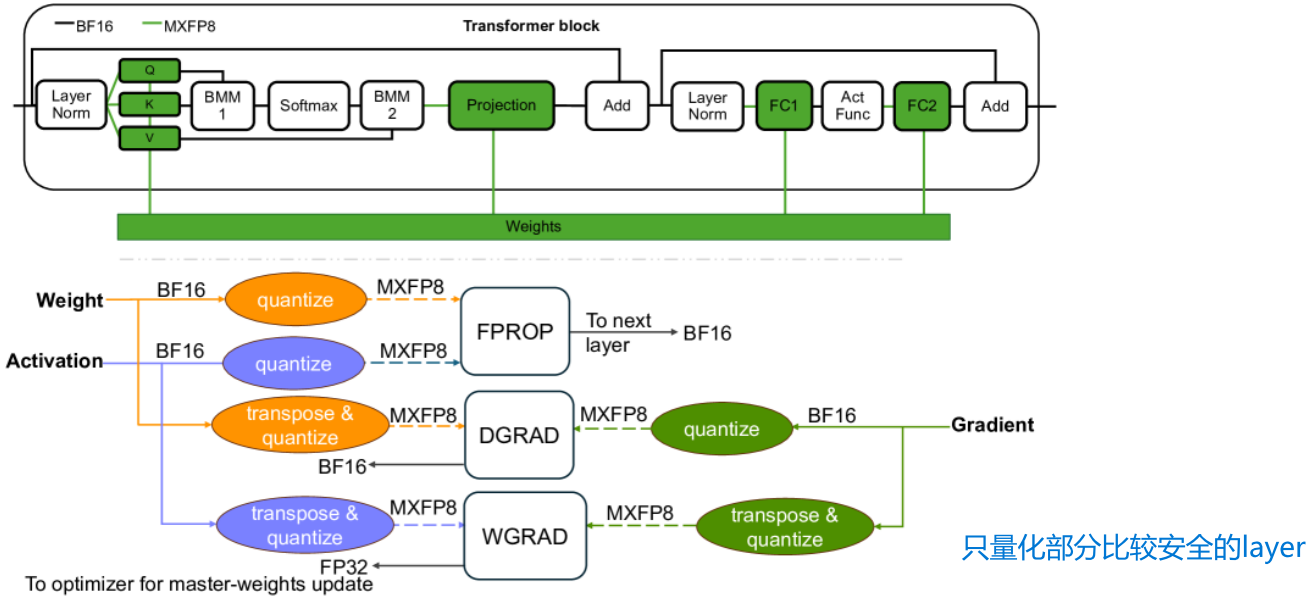
- Uses E4M3 (more mantissa) due to more scale factors
- Scale factors themselves are FP8 (E8M0), 1 per 32
- Transposes are now nontrivial!

Multiple scaling factors MXFP8 (Blackwell)





MXFP8 training in practice



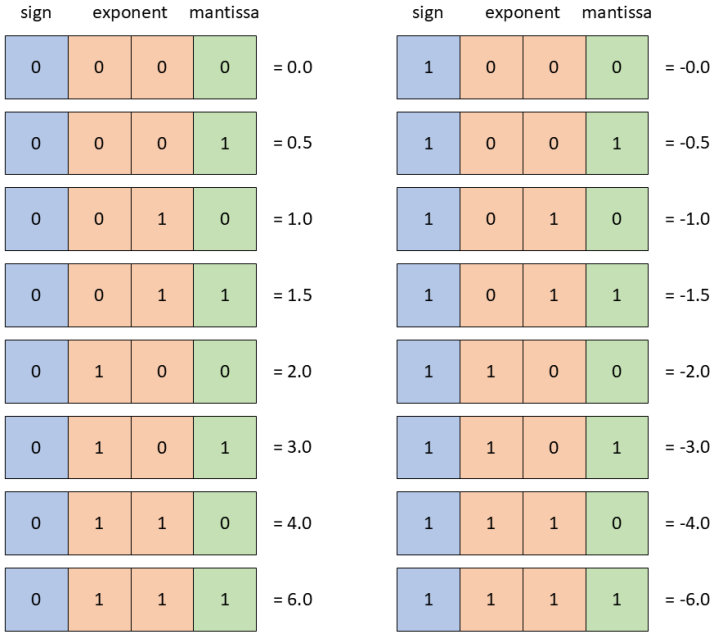
只量化部分比较安全的layer

Notice – not all weights in MXFP8, transposes also separately quantized

https://arxiv.org/html/2506.08027v2

Frontiers in low precision

MXFP4..

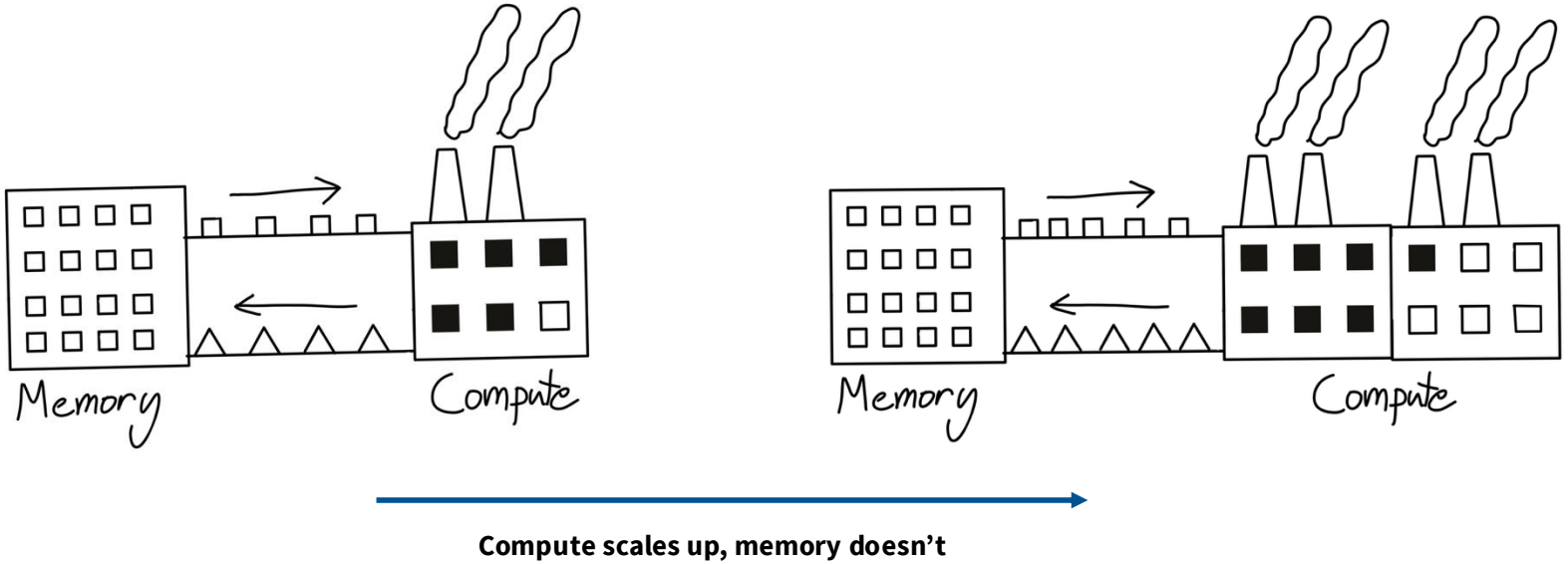


MXFP4这数据形式，总共就这几个取值

This is all the values you can represent! 1 per 16 scaling, E4M3 scaling factors.

Trick 2: Operator fusion

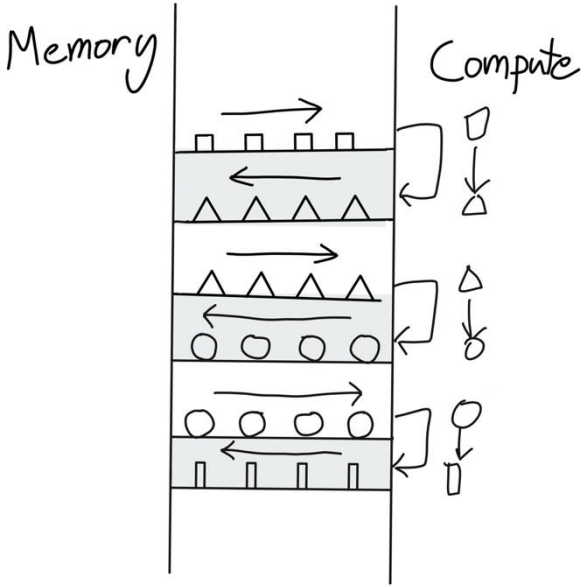
Think of a GPU like a factory – inputs come from a warehouse (memory) and is processed at a factory



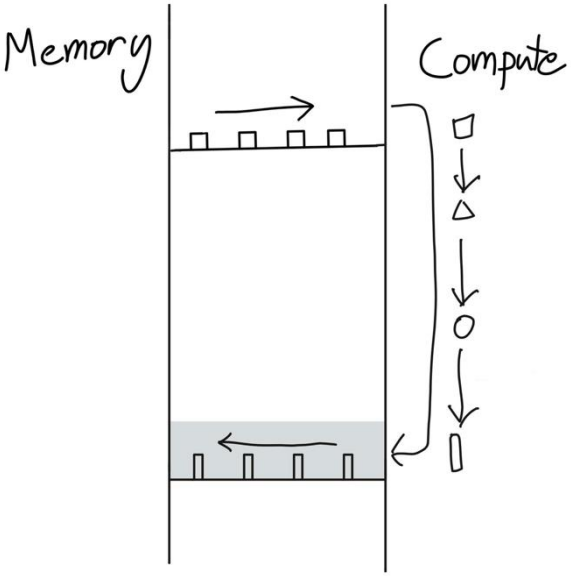
https://horace.io/brrr_intro.html

Operator fusion to minimize memory access

What if we have to do many operations? Shipping back and forth is somewhat silly



Naïve (non-fused)



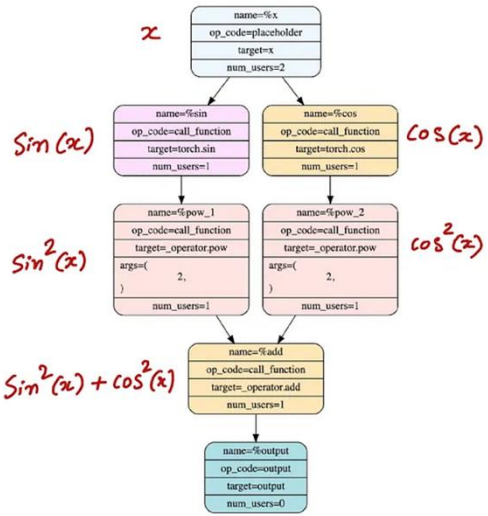
Fused kernel

Example – sines and cosines

FX GRAPH IR

```
class GraphModule(torch.nn.Module):
    def forward(self, x : torch.Tensor):
        # File: /tmp/ipykernel_2583/1502985755.py:2, code:
        sin = torch.sin(x)
        pow_1 = sin ** 2;  sin = None
        cos = torch.cos(x);  x = None
        pow_2 = cos ** 2;  cos = None
        add = pow_1 + pow_2;  pow_1 = pow_2 = None
        return (add,)
```

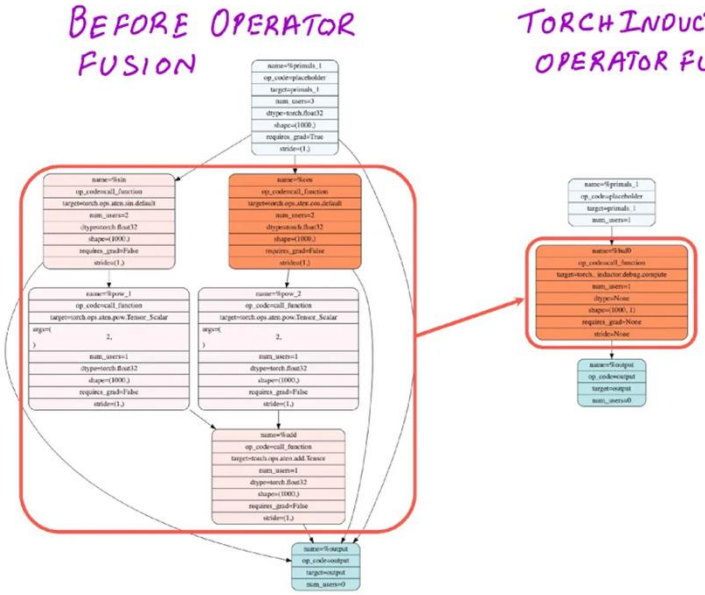
GRAPH VIZ



Computing $\sin^2 x + \cos^2 x$ naively launches 5 CUDA kernels (back and forth)

https://towardsdatascience.com/how-pytorch-2-0-accelerates-deep-learning-with-operator-fusion-and-cpu-gpu-code-generation-35132a85bd26

Fusion example

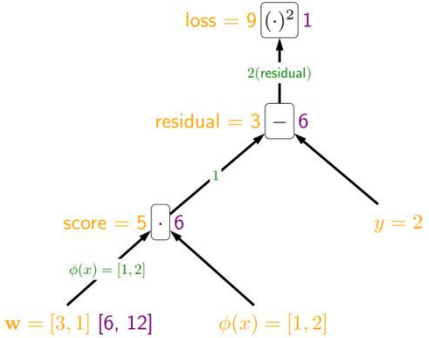


从global memory中读取一次，然后在SM中一次性做好所有计算，然后再将结果一次写入到global memory中。
而不是多次读取global memory

#TODO：torch compiler应该能自动完成这里的这种fusion对吗？torch compiler就是JAX?

All 5 pointwise operations can be fused into a single CUDA kernel call.
‘Easy’ fusions like this can be done automatically by compilers (torch.compile)

Trick 3: recomputation



$$\text{Loss}(x, y, \mathbf{w}) = (\mathbf{w} \cdot \phi(x) - y)^2$$

$\mathbf{w} = [3, 1], \phi(x) = [1, 2], y = 2$

backpropagation

$\nabla_{\mathbf{w}} \text{Loss}(x, y, \mathbf{w}) = [6, 12]$

Definition: Forward/backward values

Forward: f_i is value for subexpression rooted at i

Backward: $g_i = \frac{\partial \text{loss}}{\partial f_i}$ is how f_i influences loss

Algorithm: backpropagation algorithm

Forward pass: compute each f_i (from leaves to root)

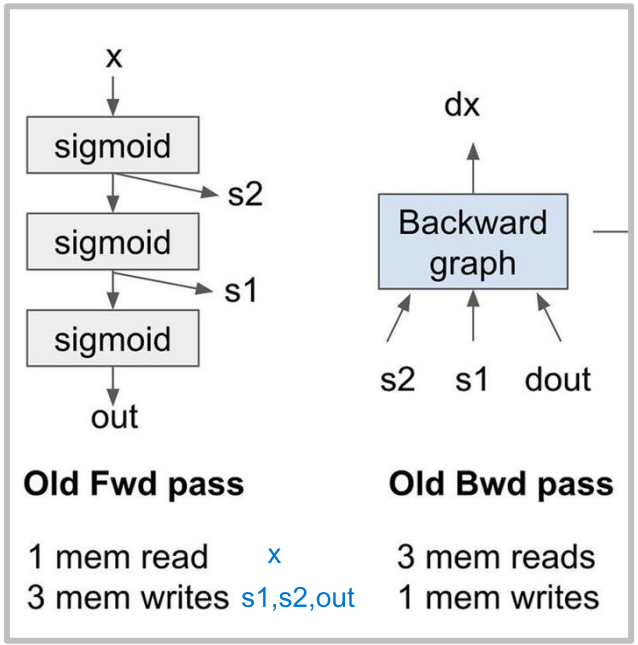
Backward pass: compute each g_i (from root to leaves)

[From cs221]

In backpropagation, we store the activations (yellow) and compute Jacobians (green)

Storing (and retrieving) activations can be expensive!

Let’s say we stack 3 sigmoids on top of each other.

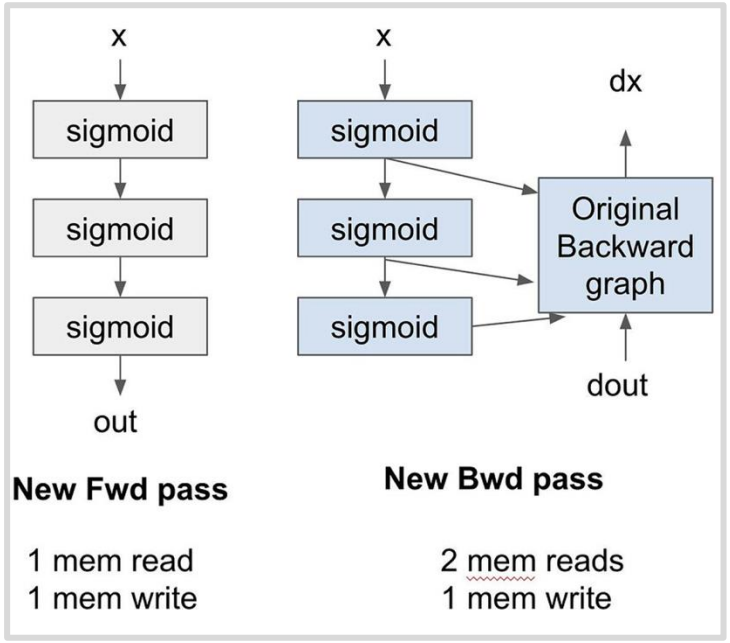


反过来了

This is really terrible for perf – 8 mem read/writes, very low arithmetic intensity.

https://dev-discuss.pytorch.org/t/min-cut-optimal-recomputation-i-e-activation-checkpointing-with-aotautograd/467

Throw away the activations, re-compute them!



#TODO：详细解释下这个例子，最好直接写出具体的计算公式和具体数值的结果，解释下到底如何通过更多的计算量换取了更少的memory访问

同时解释下，这一页和上一页ppt中，fwd和bwd的这几次read和write，分别指的是什么

这里bwd的两次read，指的是读取out和x，因为s2要通过x计算，所以需要读取x；

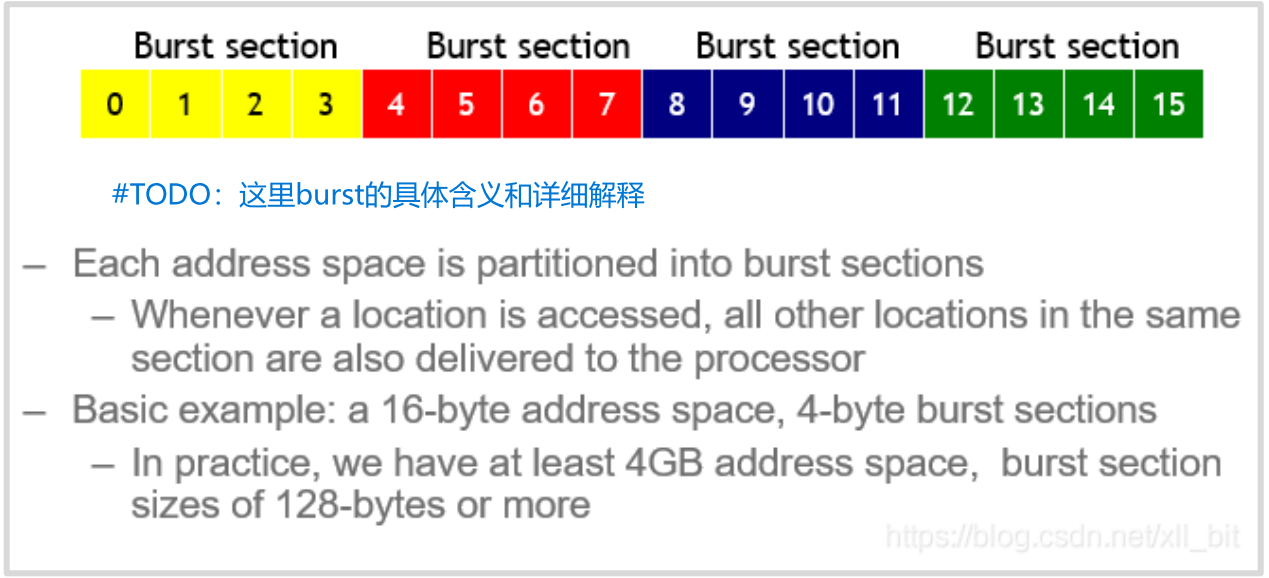
从上一页的8次内存访问，下降到了这里的五次

Throwing away computation can actually be optimal, w/ $5/8^{th}$ the memory accesses!

https://dev-discuss.pytorch.org/t/min-cut-optimal-recomputation-i-e-activation-checkpointing-with-aotautograd/467

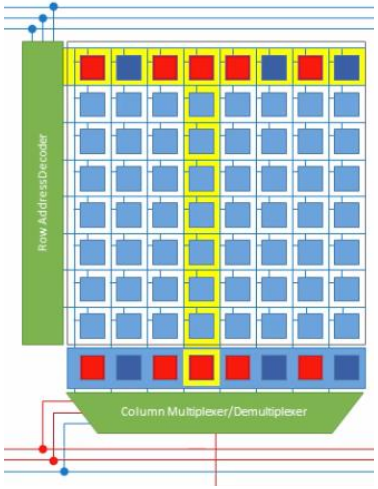
Trick (?) 4: Memory coalescing and DRAM

DRAM (global memory) is read in ‘burst mode’ – each read gives you many bytes!



[https://blog.csdn.net/xll_bit/article/details/117702476]

#TODO：amplifier指的是啥？具体是如何从DRAM上读取数据的？根据这幅图解释下。



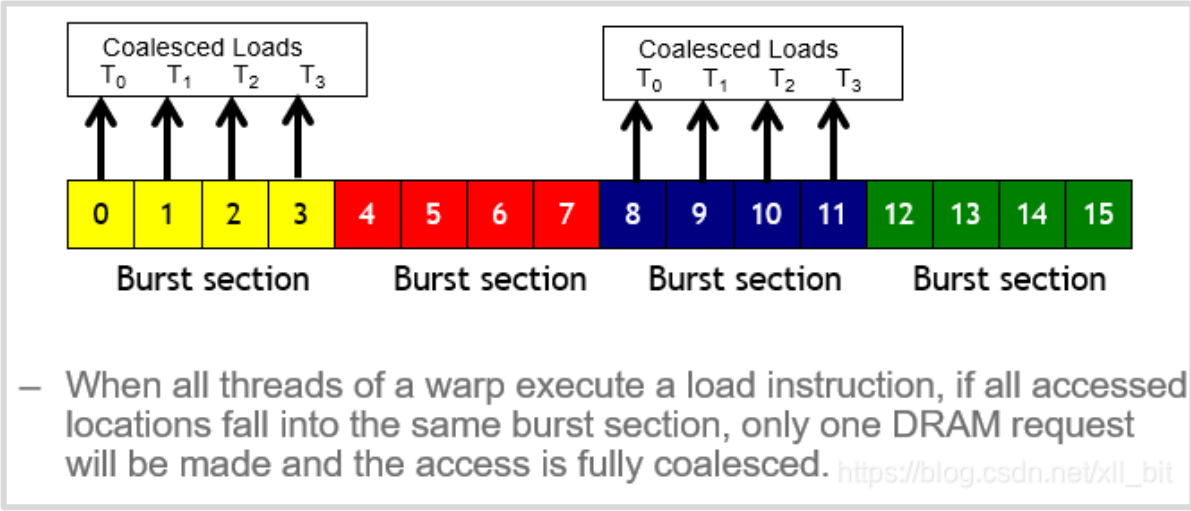
← Burst mode comes from the slow per-row copy to the sense amplifier

[https://www.youtube.com/watch?v=9BjVUmaXaCQ]

Memory coalescing

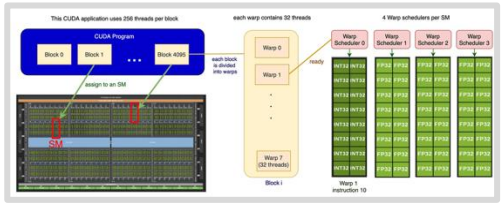
#TODO：这里coalesced的具体含义，代表的是什么意思?

Memory accesses are *coalesced* if all the threads (in a warp) fall within the same burst



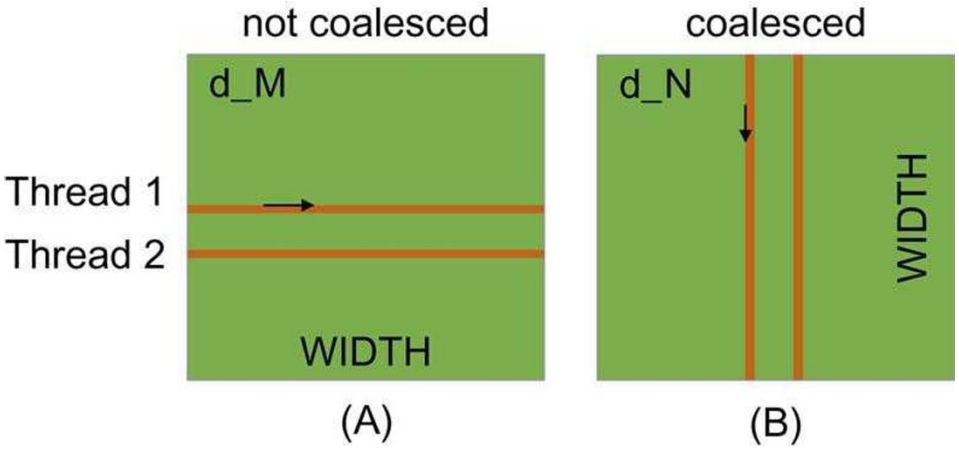
– When all threads of a warp execute a load instruction, if all accessed locations fall into the same burst section, only one DRAM request will be made and the access is fully coalesced.

https://blog.csdn.net/xll_bit



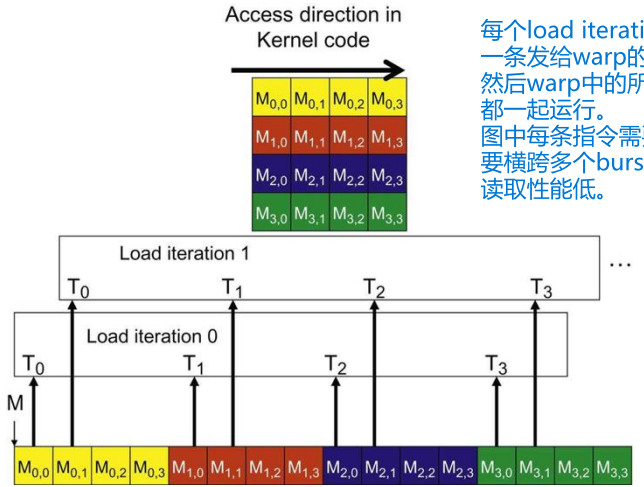
Reminder: a warp is a set of 32 consecutively numbered threads that execute together in a block. Memory accesses happen together

Coalescing for matrix multiplication



#TODO：这里图中讲的具体是什么技巧？如何运用这里得到burst，来降低memory读取？

这个图是not coalesced的坏做法



每个load iteration都代表一条发给warp的指令，然后warp中的所有thread都一起运行。
图中每条指令需要的数据要横跨多个burst，导致读取性能低。

每行都是一个burst

For row-major matrices – **threads that move along rows are not coalesced**

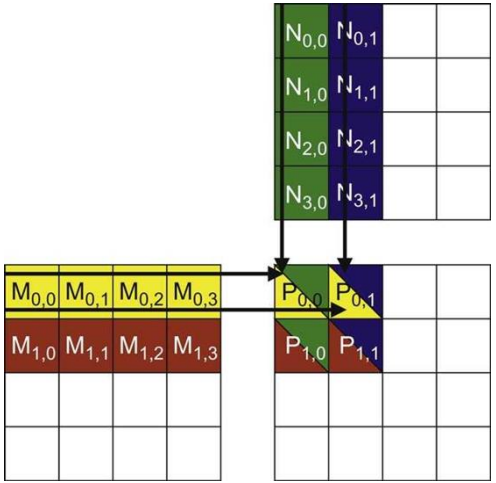
Note how the second diagram reads the entire vector at each step!

#TODO：coalesce是啥意思？这里具体讲的是什么？row-major-matrices是否指的是在内存中按照行组织的？也就是上面的右图？

Trick 5 (the big one): tiling

Tiling is the idea of grouping and ordering threads to minimize global memory access.

Let’s go back to matrix multiplication..



四个线程，每个都负责一个输出元素，这里的图例只看左上角子矩阵，包括P00,P01,P10,P11四个输出元素，对应四个thread。

Access order

$thread_{0,0}$	$M_{0,0} * N_{0,0}$	$M_{0,1} * N_{1,0}$	$M_{0,2} * N_{2,0}$	$M_{0,3} * N_{3,0}$
$thread_{0,1}$	$M_{0,0} * N_{0,1}$	$M_{0,1} * N_{1,1}$	$M_{0,2} * N_{2,1}$	$M_{0,3} * N_{3,1}$
$thread_{1,0}$	$M_{1,0} * N_{0,0}$	$M_{1,1} * N_{1,0}$	$M_{1,2} * N_{2,0}$	$M_{1,3} * N_{3,0}$
$thread_{1,1}$	$M_{1,0} * N_{0,1}$	$M_{1,1} * N_{1,1}$	$M_{1,2} * N_{2,1}$	$M_{1,3} * N_{3,1}$

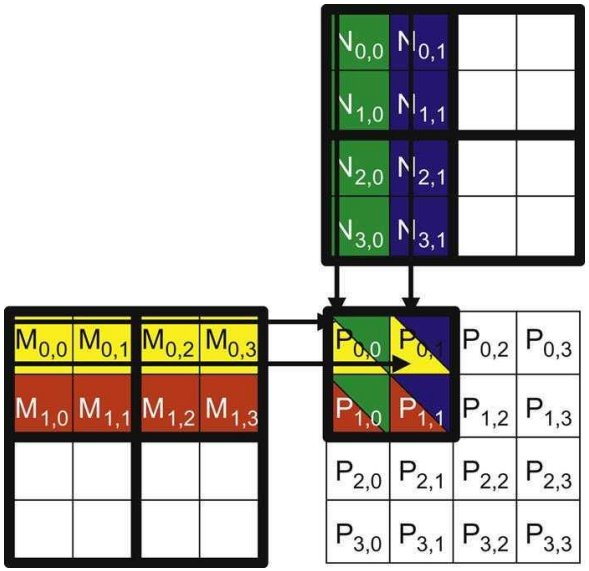
矩阵中每个元素，比如M0,0，在运算中需要用到N=4次

Note that memory access is not coalesced, and repeated (M0,0 and N1,0)

Tiling – store and reuse information in shared memory

Cut up the matrix into smaller ‘tiles’, and load this into shared memory

将矩阵切分2 * 2 = 4个分块子矩阵



这里的M00 tile指的是包含了M00元素的2*2的左上角子矩阵tile

Compute the matrix multiply in ‘phases’

1. Load $M_{0,0}$ and $N_{0,0}$ tiles into SHM
2. Compute partial sums for P

   (Done with one tile)
3. Load the $M_{0,0}$ and $N_{2,0}$ tile into SHM
4. …

SHM：shared memory

应该是M02，这里应该写错了

#TODO：按照这个例子，详细解释下这个算法。不省略地展开下具体的步骤

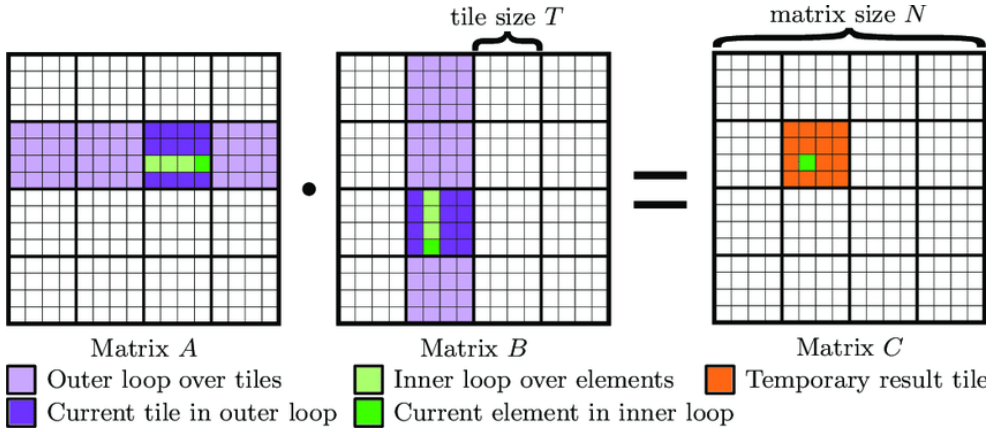
这里的M0,0。指的是

一个元素，还是左上角的这个子矩阵？每个tile指的是什么？

Advantages: repeated reads now access shared, not global memory and memory access can be coalesced

原本：M00元素需要从global memory中被取出4次，用于计算P00,P01,P02,P03；现在：M00只需要从global memory中被取出两次，每次取出都需要访问两次shared memory

Tiling math



N是矩阵的边长，这里N=16，T=4

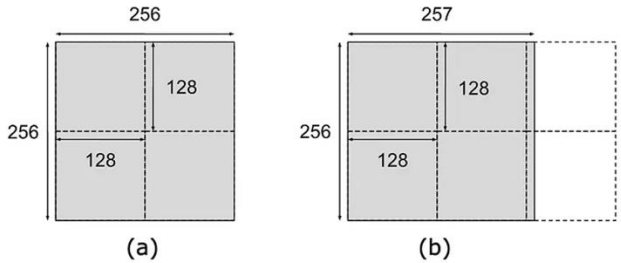
Non-tiled matrix multiply: each input is read N times from global memory

Tiled matrix multiply: each input is read $\frac{N}{T}$ times from global memory, and T times within each tile. This is a factor of T reduction in global memory access

Complexities with tiling

Tile sizes may not divide the matrix size and lead to low utilization

Figure 6. Example of tiling with 128x128 thread block tiles. (a) Best case - matrix dimensions are divisible by tile dimensions (b) Worse case - tile quantization results in six thread blocks being launched, two of which waste most of their work.



Factors affecting tile sizes

- Coalesced memory access
- Shared memory size
- Divisibility of the matrix dim

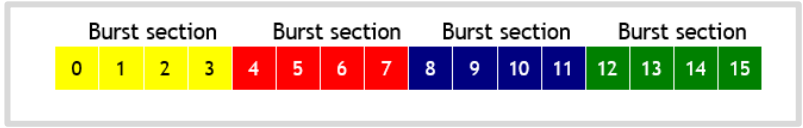
#TODO：为啥这里256 * 256 @ 256 * 257的矩阵运算，就会产生这么大的空白tile？为啥不能空白的就不tile了？我不理解底层原理。

多出的tile，会浪费一个block，白白占用一个SM的执行槽。

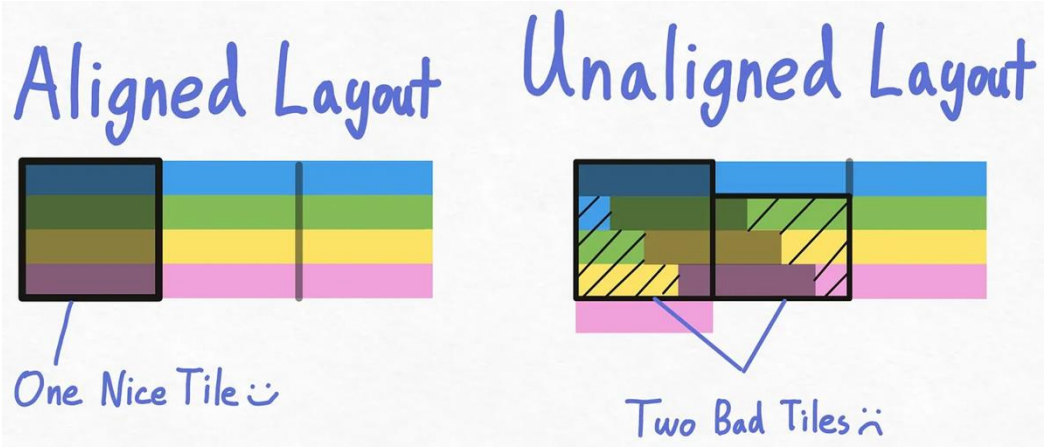
https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html#tile-quant

Complexities with tiling 2 – memory alignment

Memory comes in bursts



Loading tiles are fast if bursts align with the matrix



#TODO：完全没看懂这里。这两种情况是啥？为啥会出现这两种情况？分别什么情况下会引起？

https://www.thonking.ai/p/what-shapes-do-matrix-multiplications

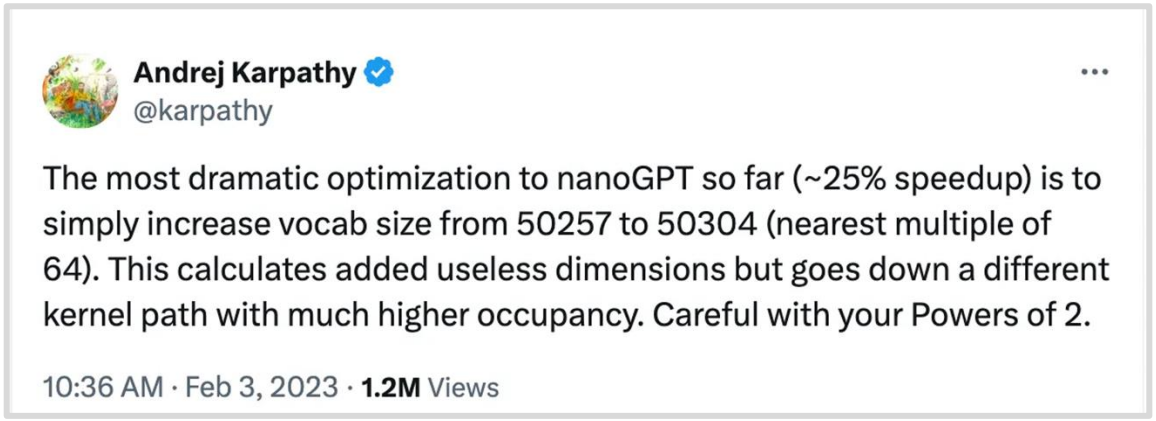
这个图说的是加载这个tile的每一行，都需要横跨两个burst

Coalesced accesses may be impossible depending on the dimension of the matrix..

(have to do padding)

每条色带（每行）代表一个矩阵的一行，包含12个元素
整个代表一个4 * 12的矩阵。被按列划分为3个tile。

Putting it together: understanding a matrix mystery



Andrej Karpathy ✓
@karpathy

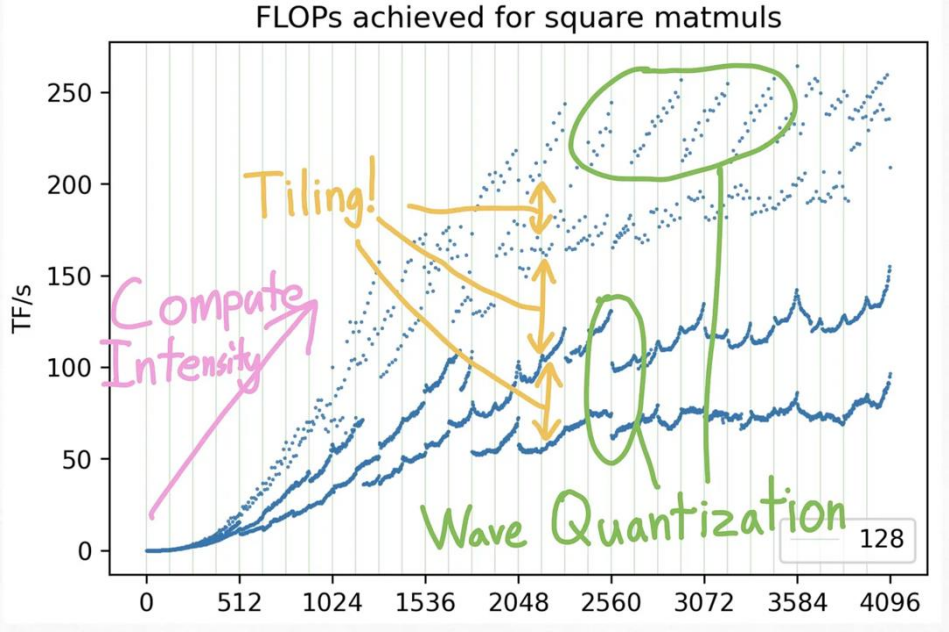
The most dramatic optimization to nanoGPT so far (~25% speedup) is to simply increase vocab size from 50257 to 50304 (nearest multiple of 64). This calculates added useless dimensions but goes down a different kernel path with much higher occupancy. Careful with your Powers of 2.

10:36 AM · Feb 3, 2023 · **1.2M** Views

Why is it *faster* to have bigger matrices?

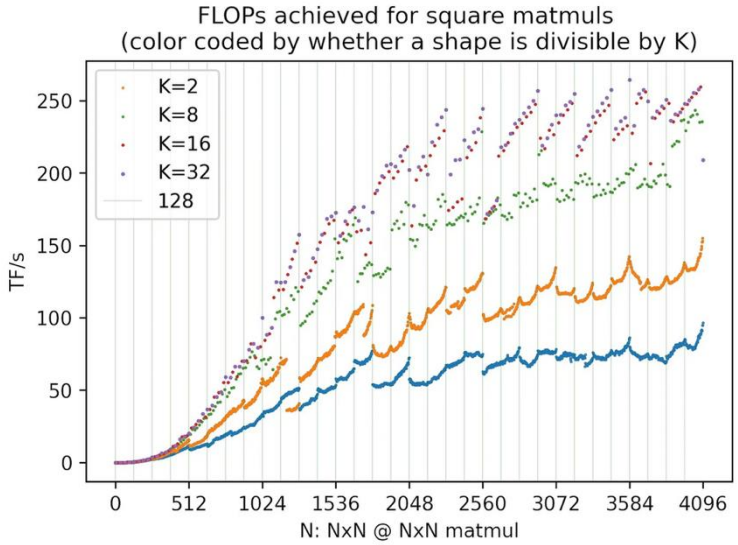
This section is from https://www.thonking.ai/p/what-shapes-do-matrix-multiplications

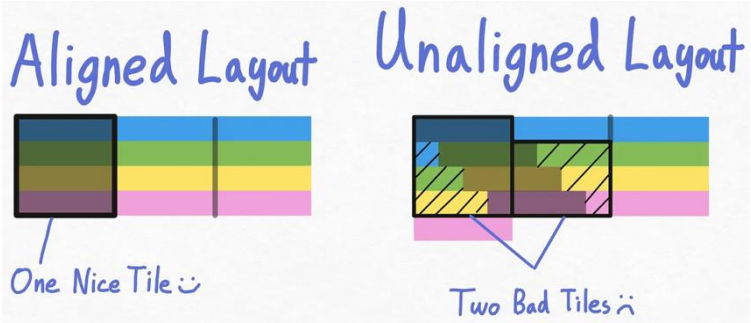
Matrix mystery



We understand some of this (compute intensity, tiling). Let’s take a closer look..

Part 1: tiling



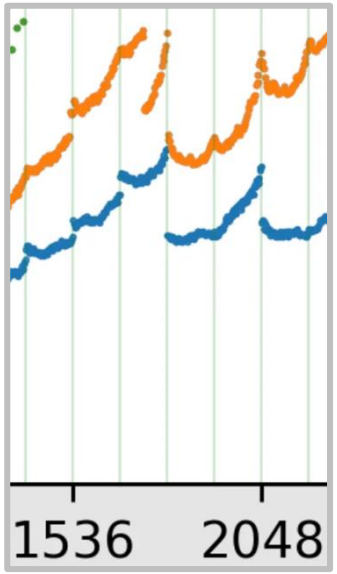


对于太小的维度的矩阵相乘，比如16或者32，此时比burst window更小，所以两者其实TF/s差不多

Tiling has a major impact through alignment.

Part 2: wave quantization

What's with the periodic behavior?



This happens at 1792 to 1793 size.

Why? Using a tile size of 256×128, there are

$$\frac{1792}{256} \times \frac{1792}{128} = 7 \times 14 = 98$$

tiles. If we increase this to 1793, we have

$$8 \times 15 = 120$$

tiles.

An A100 has 108 SMs, so it cannot execute all 120

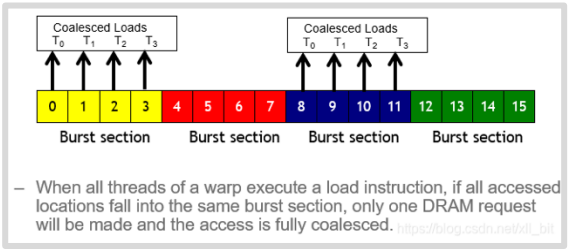
#TODO：详细解释下这个例子。为啥之前是98个tile，之后是120个？以及和SM的数量有啥关系？

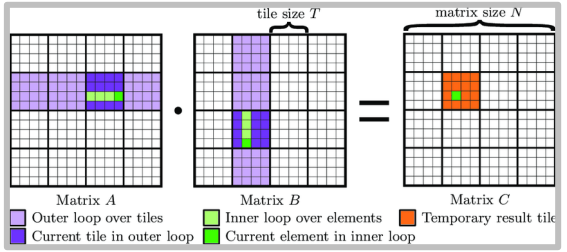
Recap of part 2: making ML workloads go fast

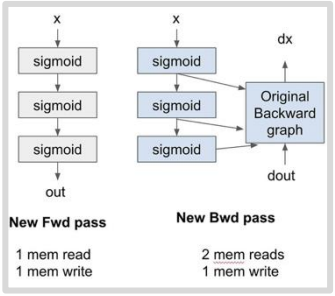
- Reduce memory accesses
  - Coalescing
  - Fusion
- Move memory to shared memory
  - Tiling

#TODO：这里的shard memory指的是啥?

- Trade memory for compute/accuracy
  - Quantization
  - Recomputation

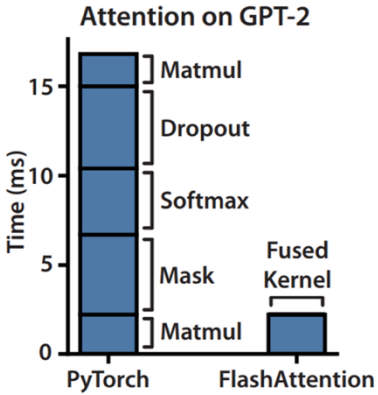




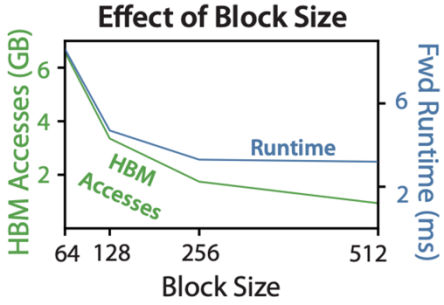


Part 3: Using what we know to understand Flash Attention

Flash attention [Dao et al] dramatically accelerates attention.. But how?



Attention	Standard	FLASHATTENTION
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3

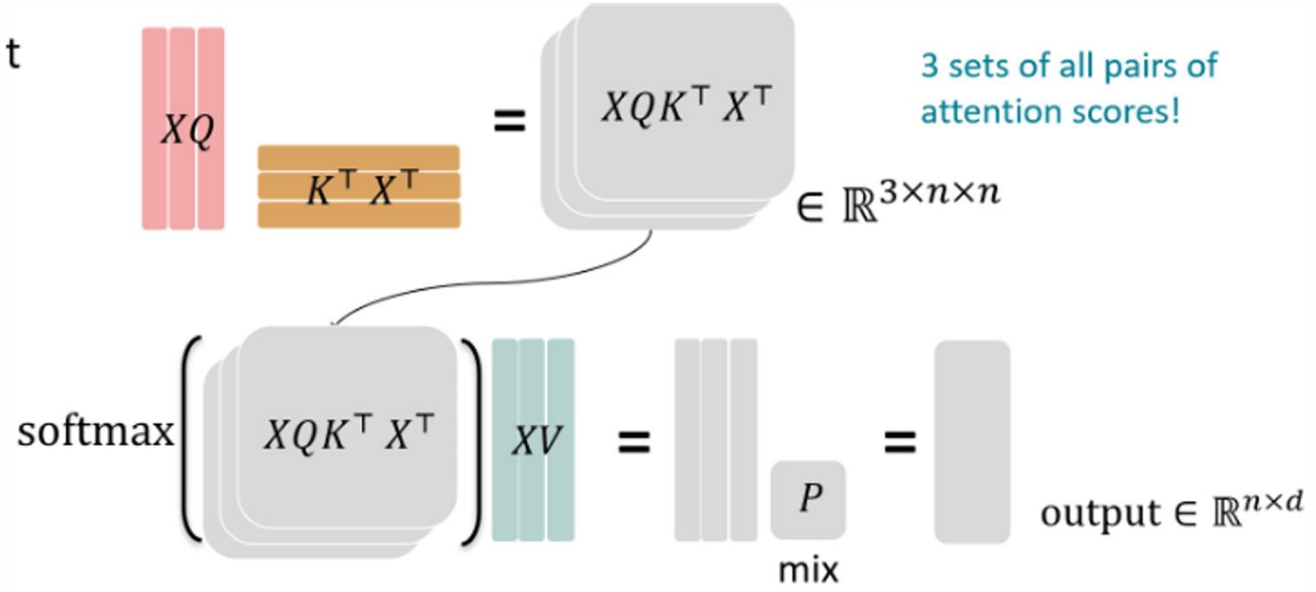


Technique from paper:

We apply two established techniques (tiling, recomputation) to overcome the technical challenge of computing exact attention in sub-quadratic HBM accesses. We describe this in Algorithm 1. The main idea

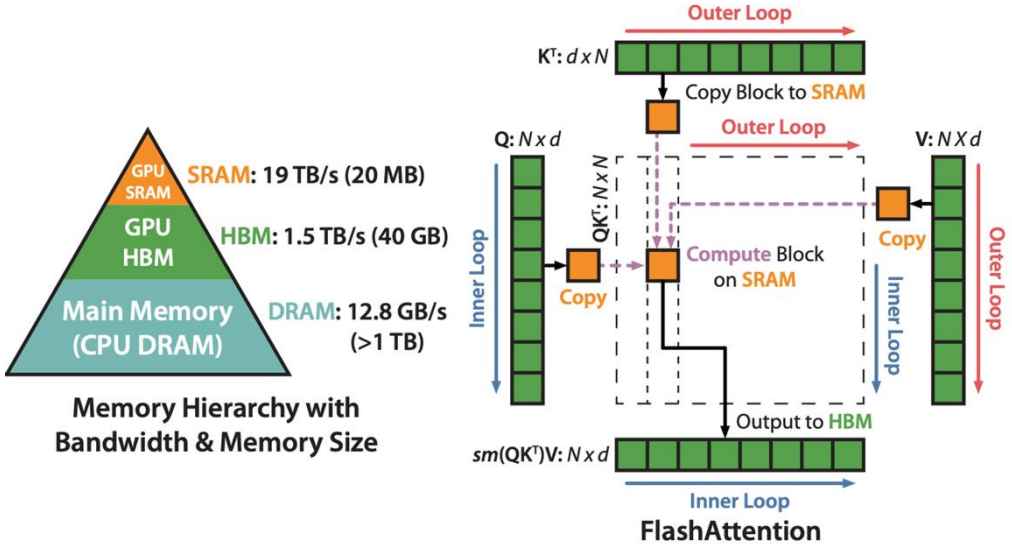
Recap of attention computation

Attention computation: 3 matrix multiplies (K, Q, V) with a softmax in between



X是输入，K,Q,V是就是Wk，Wq和Wv矩阵

Tiling part 1: tiling for the KQV matrix multiply



This figure 1 from the paper is literally just tiling for a KQV matrix multiply..

But what do we do about the softmax?

#TODO：根据这里的图例，详细解释下flash attn，最好使用伪代码的形式

Tiling part 2: incremental computation of the softmax

From Mikailov and Gimelshein 2018,

Normal softmax

$$y_i = \frac{e^{x_i - \max_{k=1}^{V} x_k}}{\sum_{j=1}^{V} e^{x_j - \max_{k=1}^{V} x_k}} \tag{2}$$

All major DL frameworks are using this safe version for the Softmax computation: TensorFlow

Algorithm 2 Safe softmax

1: $m_0 \leftarrow -\infty$
2: **for** $k \leftarrow 1, V$ **do**
3: $\quad m_k \leftarrow \max(m_{k-1}, x_k)$
4: **end for**
5: $d_0 \leftarrow 0$
6: **for** $j \leftarrow 1, V$ **do**
7: $\quad d_j \leftarrow d_{j-1} + e^{x_j - m_V}$
8: **end for**
9: **for** $i \leftarrow 1, V$ **do**
10: $\quad y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$
11: **end for**

Online softmax

Algorithm 3 Safe softmax with online normalizer calculation

1: $m_0 \leftarrow -\infty$
2: $d_0 \leftarrow 0$
3: **for** $j \leftarrow 1, V$ **do**
4: $\quad m_j \leftarrow \max(m_{j-1}, x_j)$
5: $\quad d_j \leftarrow d_{j-1} \times e^{m_{j-1} - m_j} + e^{x_j - m_j}$
6: **end for**
7: **for** $i \leftarrow 1, V$ **do**
8: $\quad y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$
9: **end for**

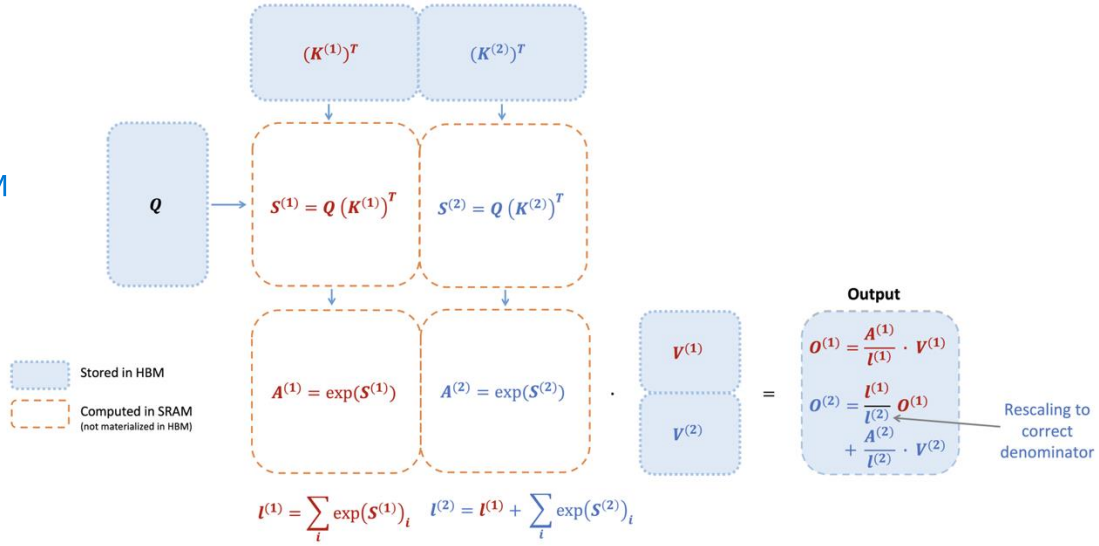
To keep track of the max, incrementally update the max, and set up a telescoping sum

This lets you compute the softmax tile-by-tile

#TODO：这里能否和这个lecture之前讲的内容结合一下，详细说明优化点以及为啥能提速

Putting it all together – the forward pass of flash attention

#TODO：详细解释下这幅图。
为啥一些是放在SRAM中，
一些是放在DRAM中?
其中not materialized in HBM
是啥意思?



From Dao 2023, we see

- Tile-wise computation of the inner products, (S)
- Fusion of the exponential operator
- Tile-wise computation of the softmax via the online, telescoping sum trick

(We won't cover the backward pass – but they recompute tile-by-tile..)

Recap for the whole lecture

- Hardware powers scale, and low-level details determine what scales or doesnt
- Curent GPU based compute strongly encourages thinking about matmul + data movement
- Thinking carefully about the GPU (coalescing, tiling, fusion) leads us to good performance

